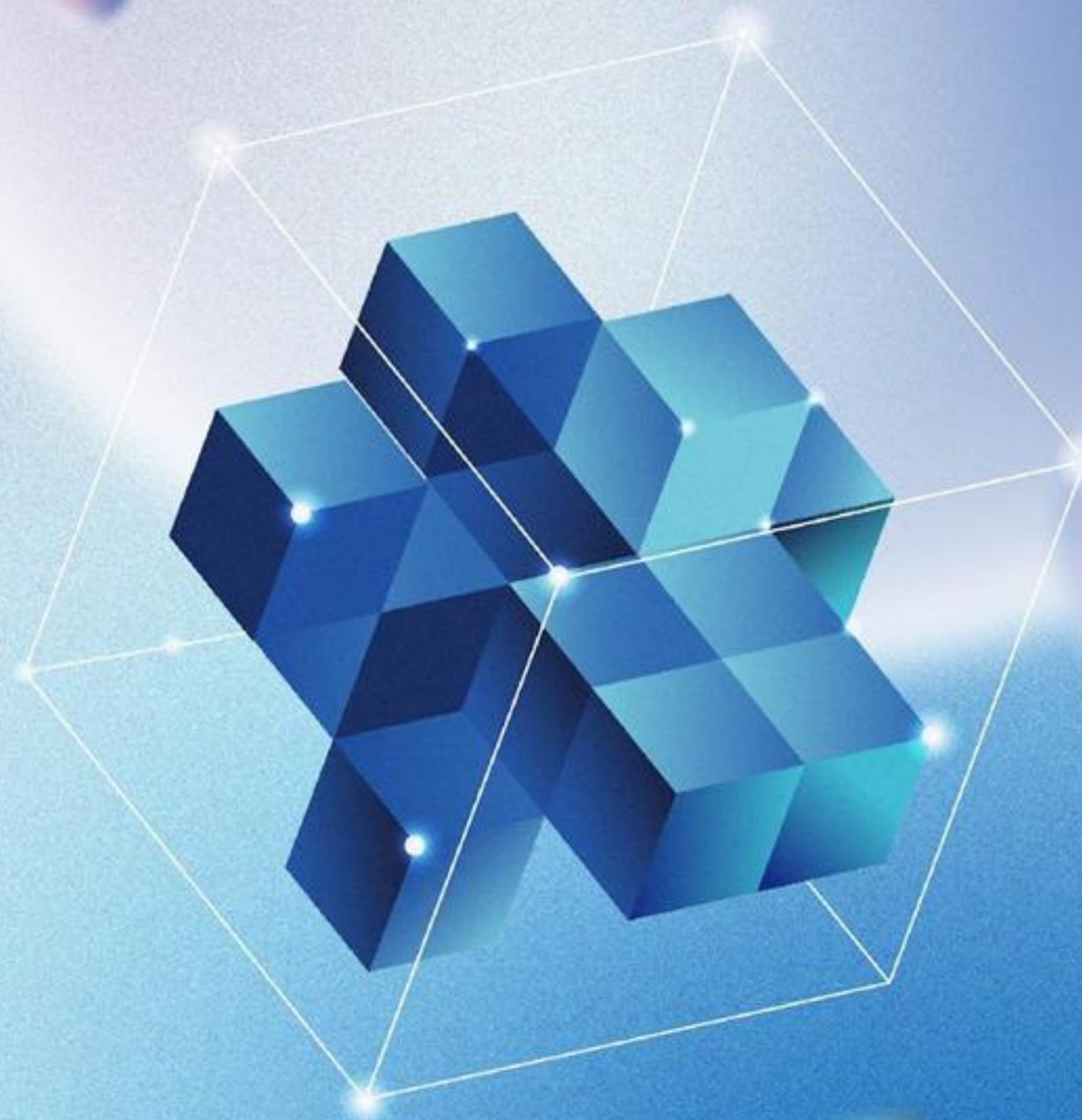


사무계 2주차

AI 모델 / RAG



CONTENTS

01 머신러닝 / 딥러닝

머신러닝

딥러닝

03 랭체인

랭체인 기초

랭체인 응용

05 RAG 챗봇

RAG 챗봇 구축

RAG 챗봇 평가

02 트랜스포머 / 허깅페이스

트랜스포머 모델

허깅페이스

04 라마인덱스

라마인덱스 기초

라마인덱스 응용

머신러닝



머신러닝이란?

데이터를 학습해 패턴을 발견하고 예측하도록 하는 인공지능



머신러닝 종류

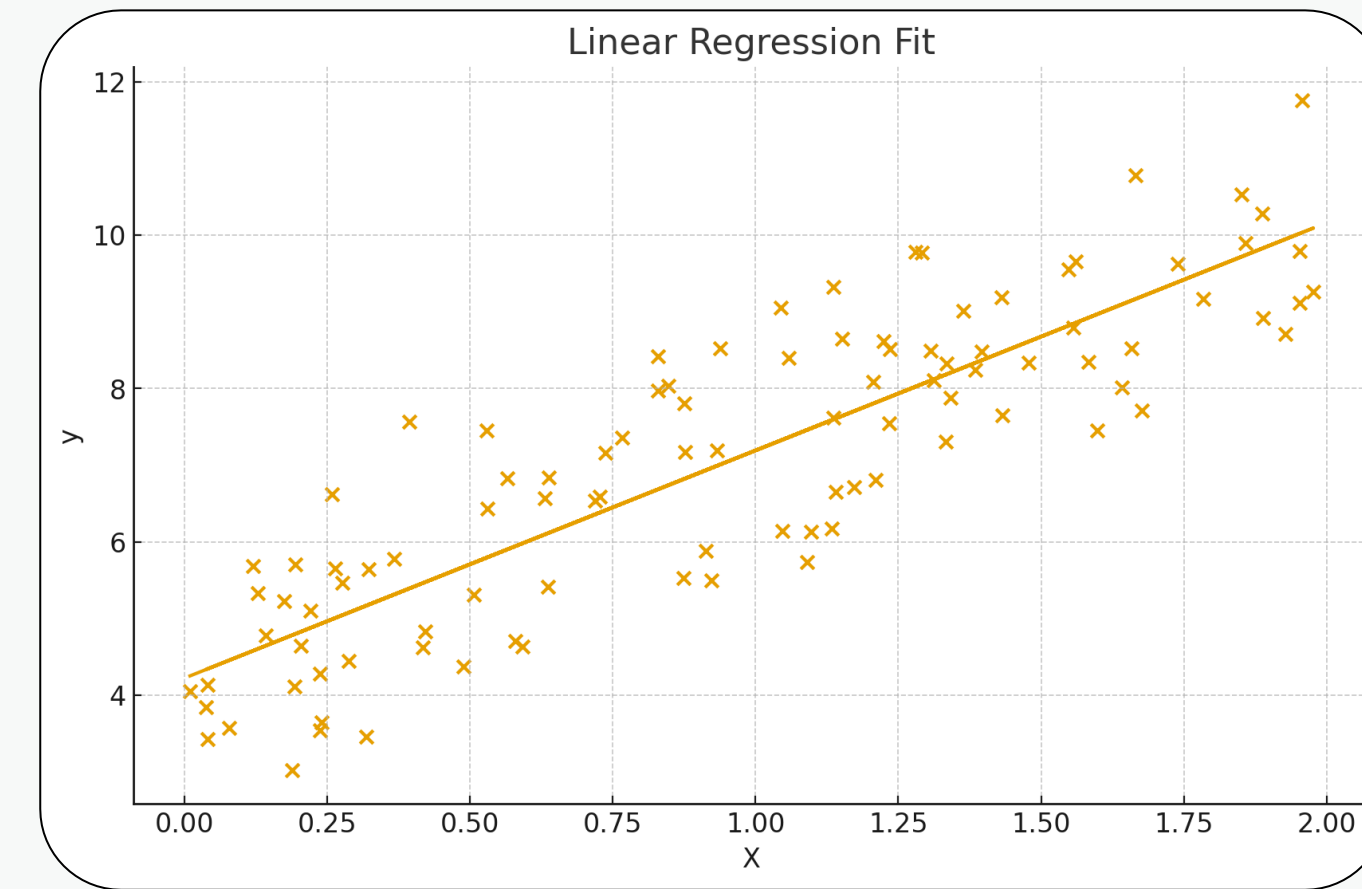
구분	설명	예시
지도학습	입력과 정답(라벨)이 함께 있는 데이터로 학습	스팸 메일 분류, 가격 예측
비지도학습	정답이 없는 데이터를 통해 구조 분석	군집 분석, 차원 축소
강화학습	환경과 상호작용하며 보상 최대화하도록 학습	게임 AI, 로봇 제어

지도학습

- 회귀 (Regression)

연속적인 값을 예측하는 문제 (숫자 예측)

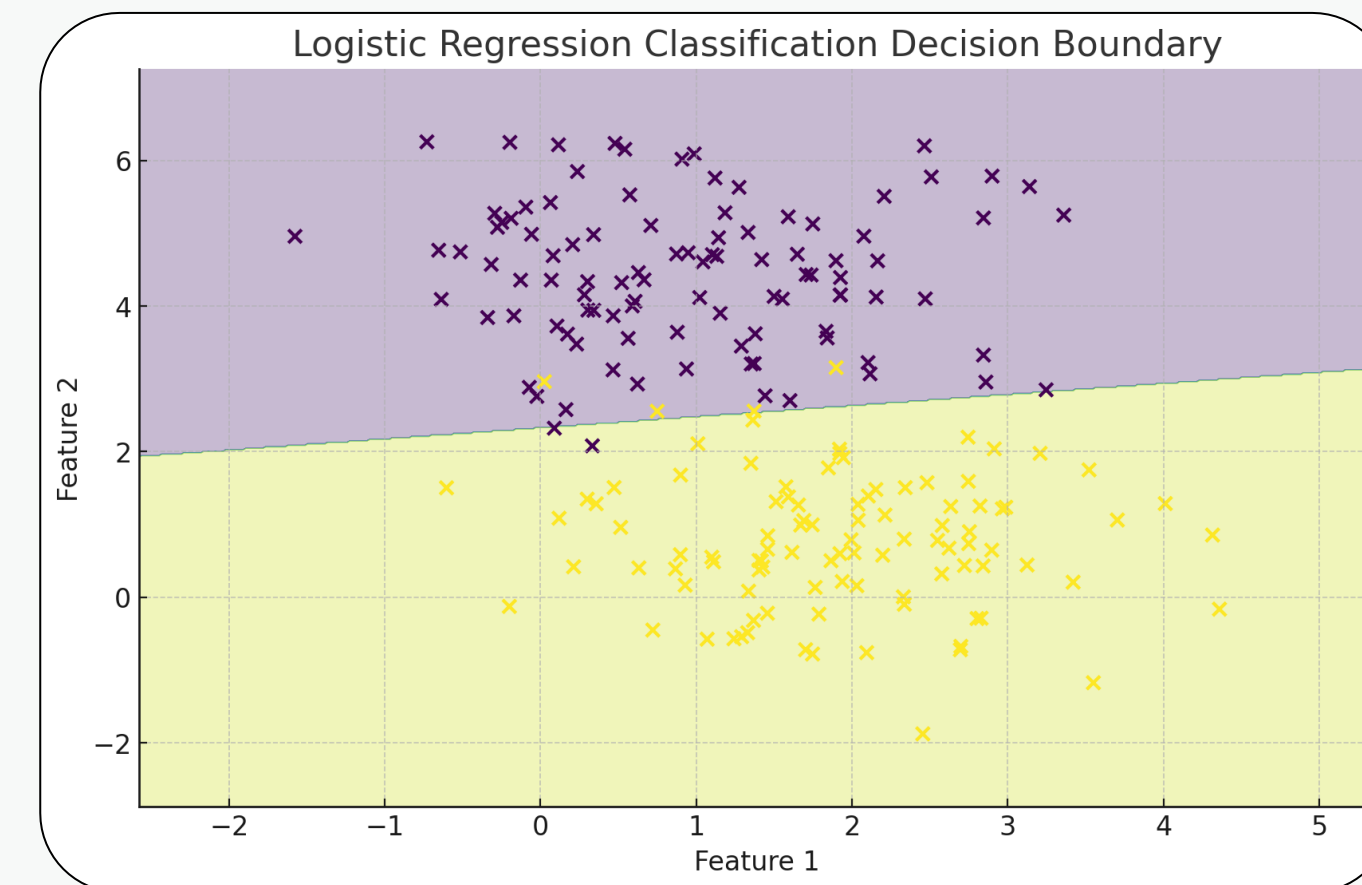
예: 집값, 주가 예측



- 분류 (Classification)

범주형 레이블을 예측하는 문제 (범주 예측)

예: 질병 여부, 이메일 스팸 판별



머신러닝 프로세스





데이터 전처리 - 1

작업	설명
중복 제거	drop _ duplicates() 등을 사용하여 동일한 데이터 행 제거
결측치 처리	삭제(제거), 평균/중앙값 대체, 예측 모델 기반 대체
이상치 탐지/처리	IQR, z-score, boxplot 등으로 이상치 탐지 후 처리
상관관계 분석	피어슨 상관계수 계산 및 heatmap 시각화

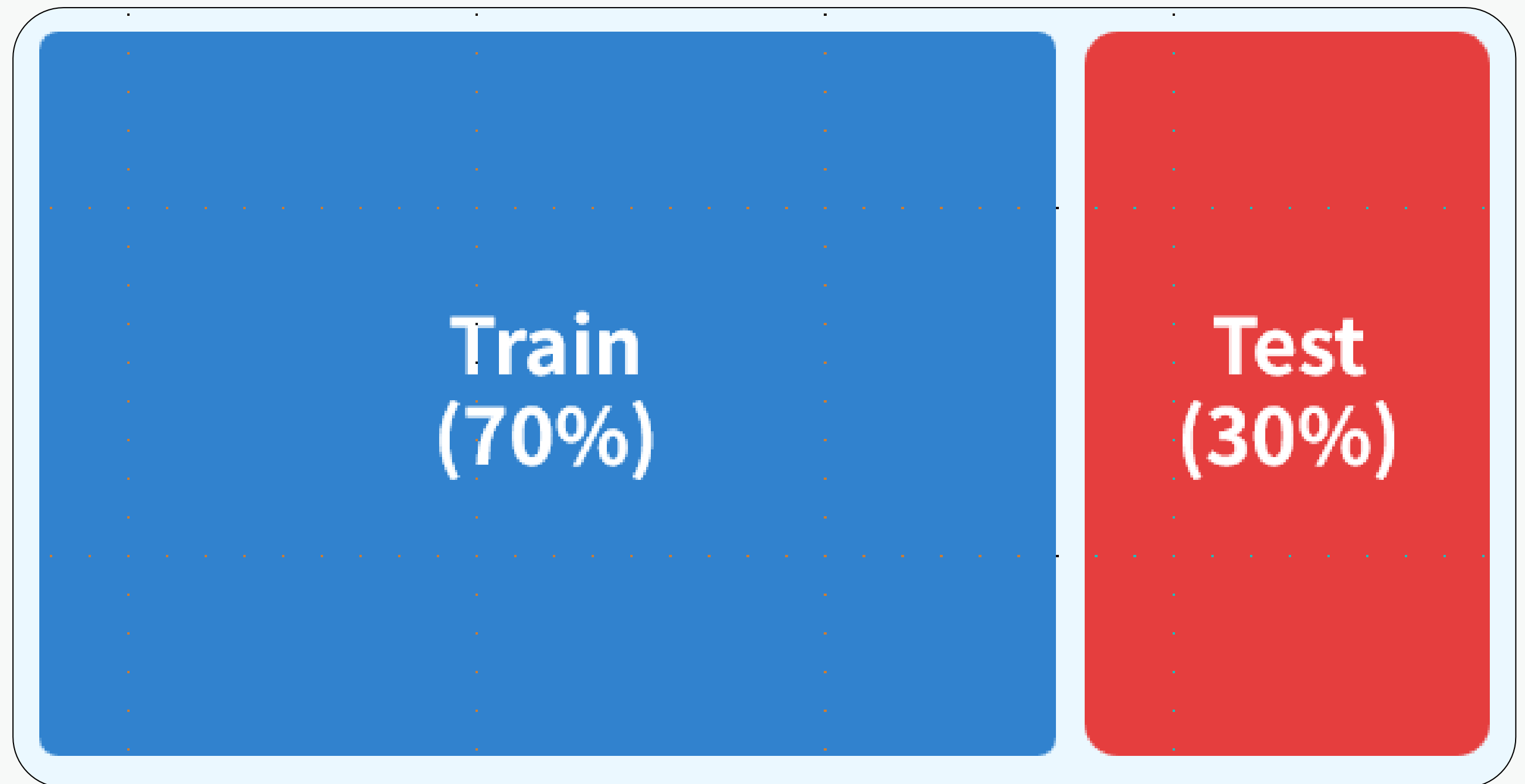


데이터 전처리 - 2

작업	설명
타겟 변수 분석	회귀: 값 분포 / 분류: 클래스 불균형 여부 확인
인코딩	범주형 → 숫자형 변환 (Label Encoding, One-Hot Encoding 등)
스케일링	정규화(Min-Max), 표준화(StandardScaler)로 값 범위 조정
파생 변수 생성	날짜에서 년/월/일 분리, 구간화 등 새로운 특징 추가

❏ 데이터 분할

- 데이터셋을 학습용(Train)과 테스트용(Test)으로 분할합니다.
- 보통 70:30 또는 80:20
- `train_test_split()` 사용



회귀 모델 - 선형회귀 (Linear Regression)

- 하나의 독립 변수(X)와 종속 변수(y) 간의 선형 관계를 찾아 예측함
- 직선 형태로 예측 (1차 함수)
- 과적합 위험이 적지만 비선형 데이터를 예측하는 데 한계가 있음

$$y = \beta_0 + \beta_1 x + \epsilon$$

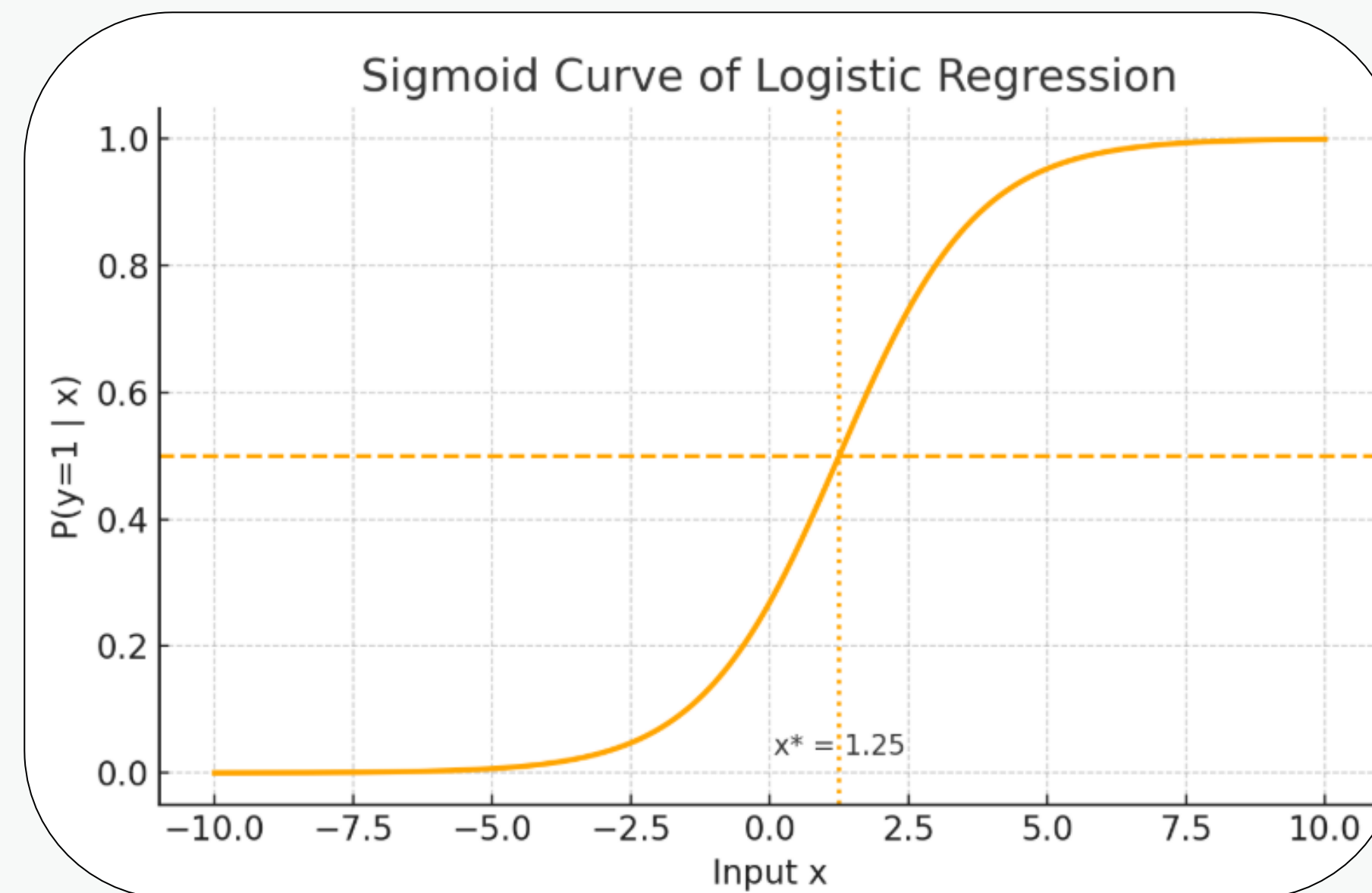
회귀 모델 - 다중 선형회귀 (Multiple Linear Regression)

- 2개 이상의 독립 변수($X_1, X_2, \dots, X_n$)를 사용하여 y 와의 선형 관계를 추정함.
- 다양한 feature의 영향력을 분석 가능
- 비선형 데이터를 예측하는 데 한계가 있음

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_n x_n + \epsilon$$

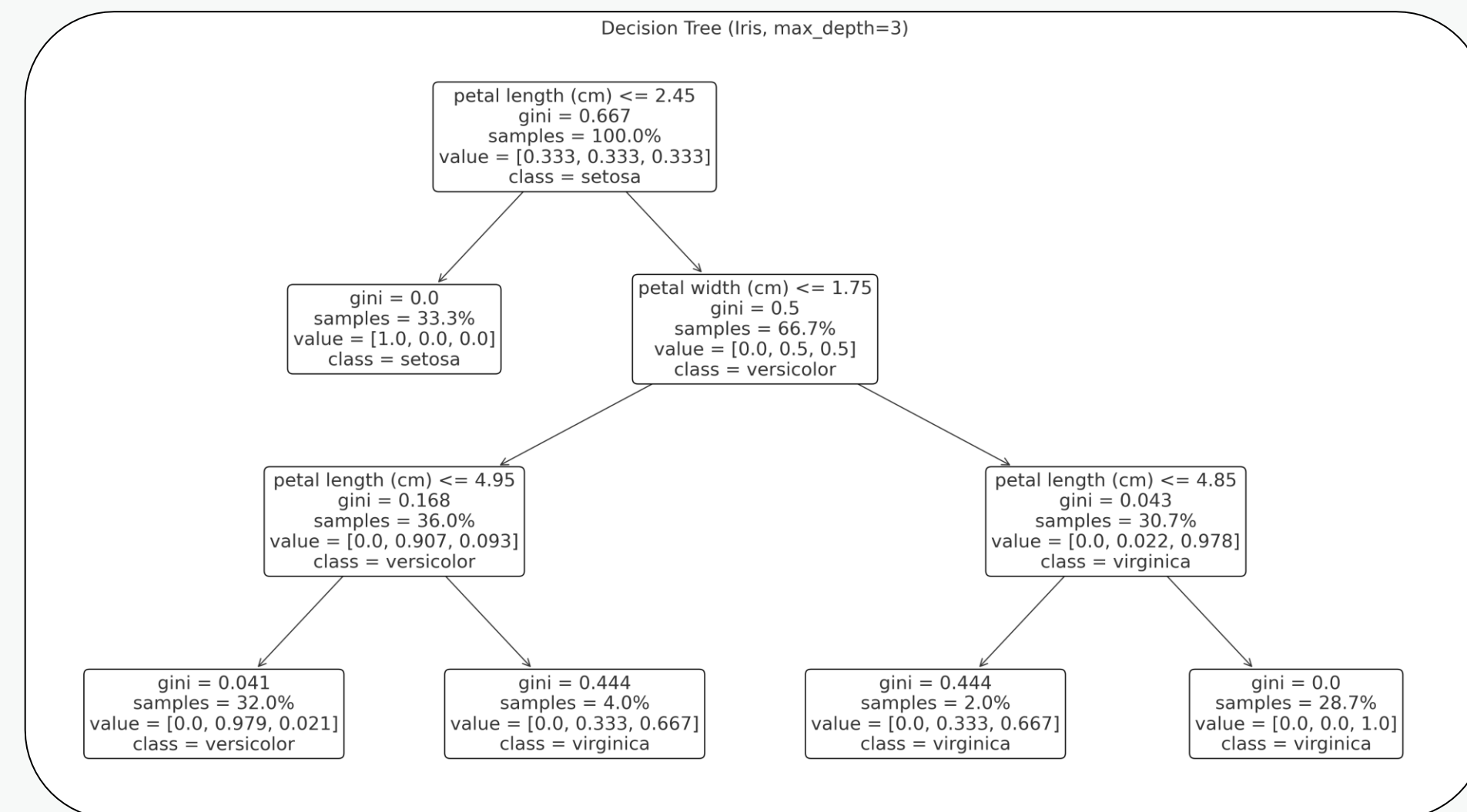
분류모델 - 로지스틱회귀

- 연속형 입력 변수를 사용하지만 출력값은 이진(0 또는 1) 형태로 예측함
- 시그모이드 함수(로지스틱 함수)를 이용해 선형 결합 결과를 확률 값(0~1)로 변환함
- 분류 문제에 활용됨



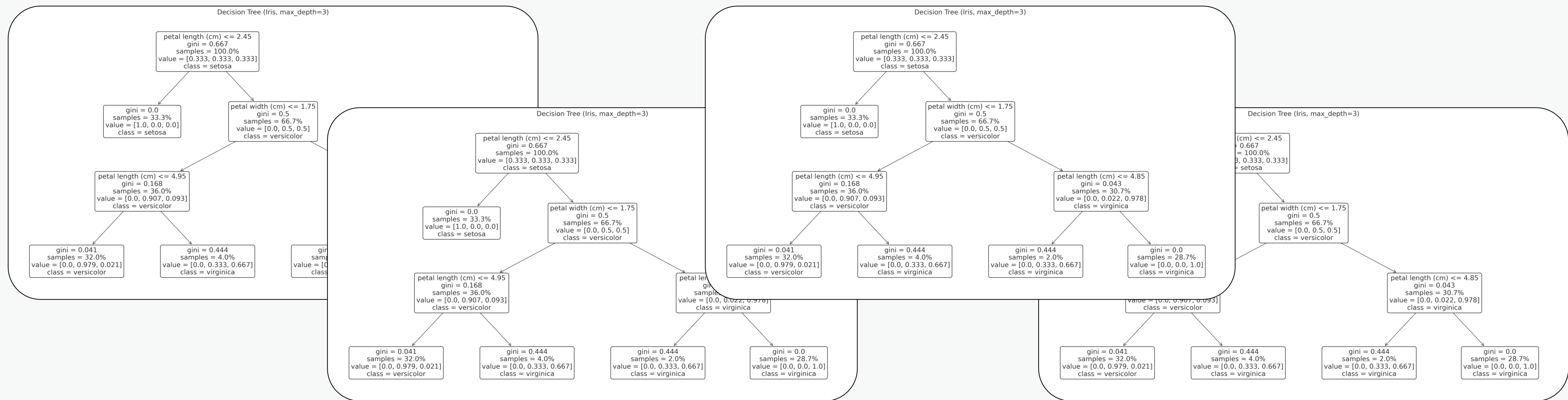
결정트리 (Decision Tree)

- 데이터를 특징값에 따라 조건 분기하여 예측하는 트리 구조의 모델
- 노이즈에 민감 → 과적합 가능성 있음
- 변수 간 상호작용 및 비선형 관계 학습 가능



랜덤포레스트 (Random Forest)

- 여러 개의 결정트리를 생성한 뒤, 각 트리의 결과를 평균(회귀) 또는 투표(분류)로 결합하는 앙상블 모델
- 과적합 위험이 낮아 안정적인 성능 제공

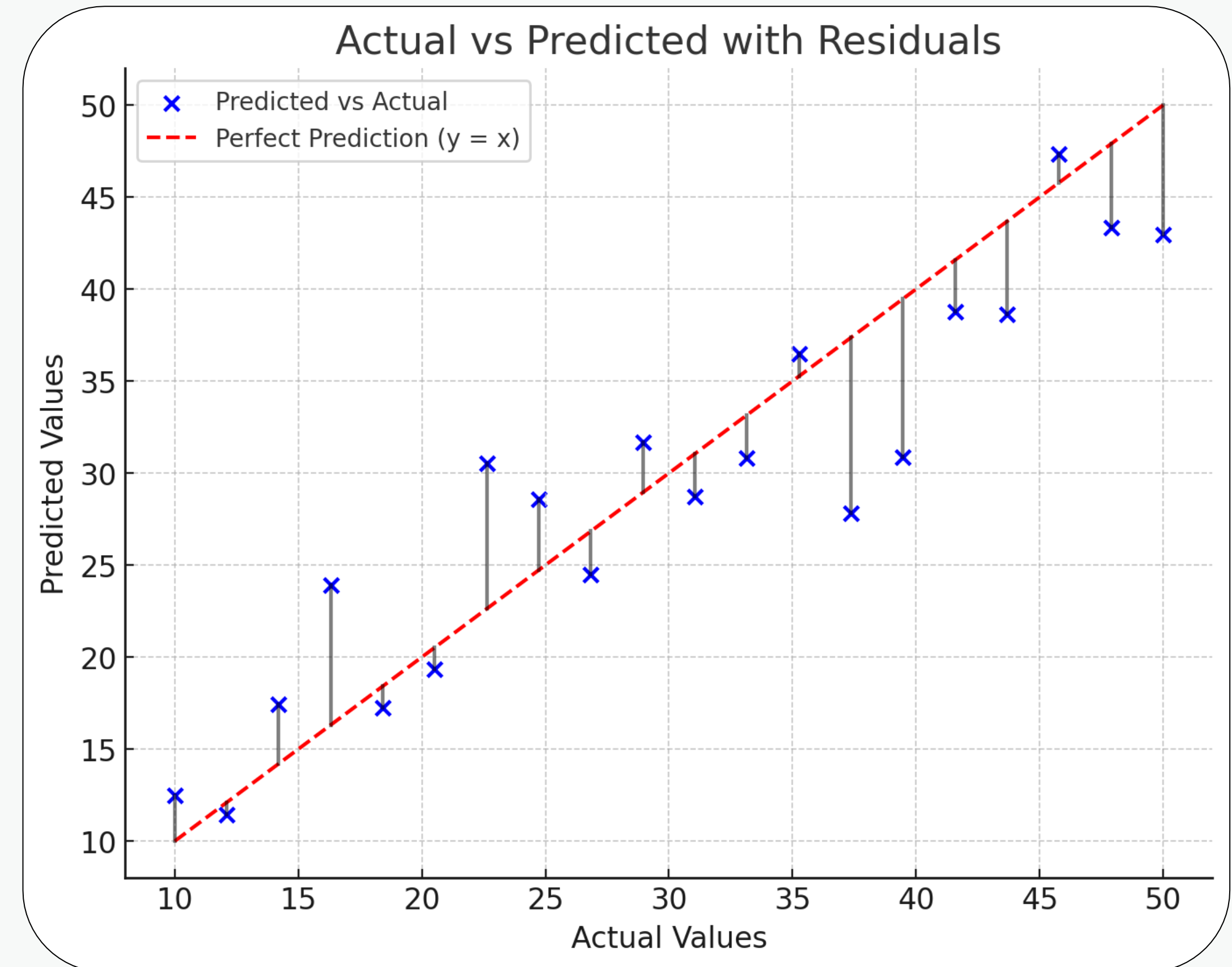


XGBoost (Extreme Gradient Boosting)

- 부스팅(Boosting) 기법을 기반으로 한 고성능 앙상블 모델. 이전 모델의 오차를 보완하면서 트리를 순차적으로 학습
- Gradient Boosting 기반 → 높은 예측 / 성능정규화(L1, L2)를 통해 과적합 방지

회귀 모델 평가 지표

- MAE (평균 절대 오차)
- MSE (평균 제곱 오차)
- RMSE (평균 제곱근 오차)
- R^2 (결정 계수)



RSS (잔차 제곱합)

- 실제값과 예측값의 차이(잔차) 를 제곱하여 모두 더한 값
- 모델이 데이터를 얼마나 잘 설명하는지 나타내는 오차의 총량

$$RSS = \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

MAE (평균 절대 오차)

- 실제값과 예측값의 차이의 절대값의 평균
- 이상치에 둔감한 모델 성능 비교할 때 적합

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

MSE (평균 제곱 오차)

- 오차 제곱의 평균 → 큰 오차에 더 큰 패널티
- 이상치가 있을 때 모델이 민감하게 반응

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

RMSE (평균 제곱근 오차)

- MSE의 제곱근 → 원래 데이터 단위와 동일
- 예측 오차 크기를 직관적으로 파악

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

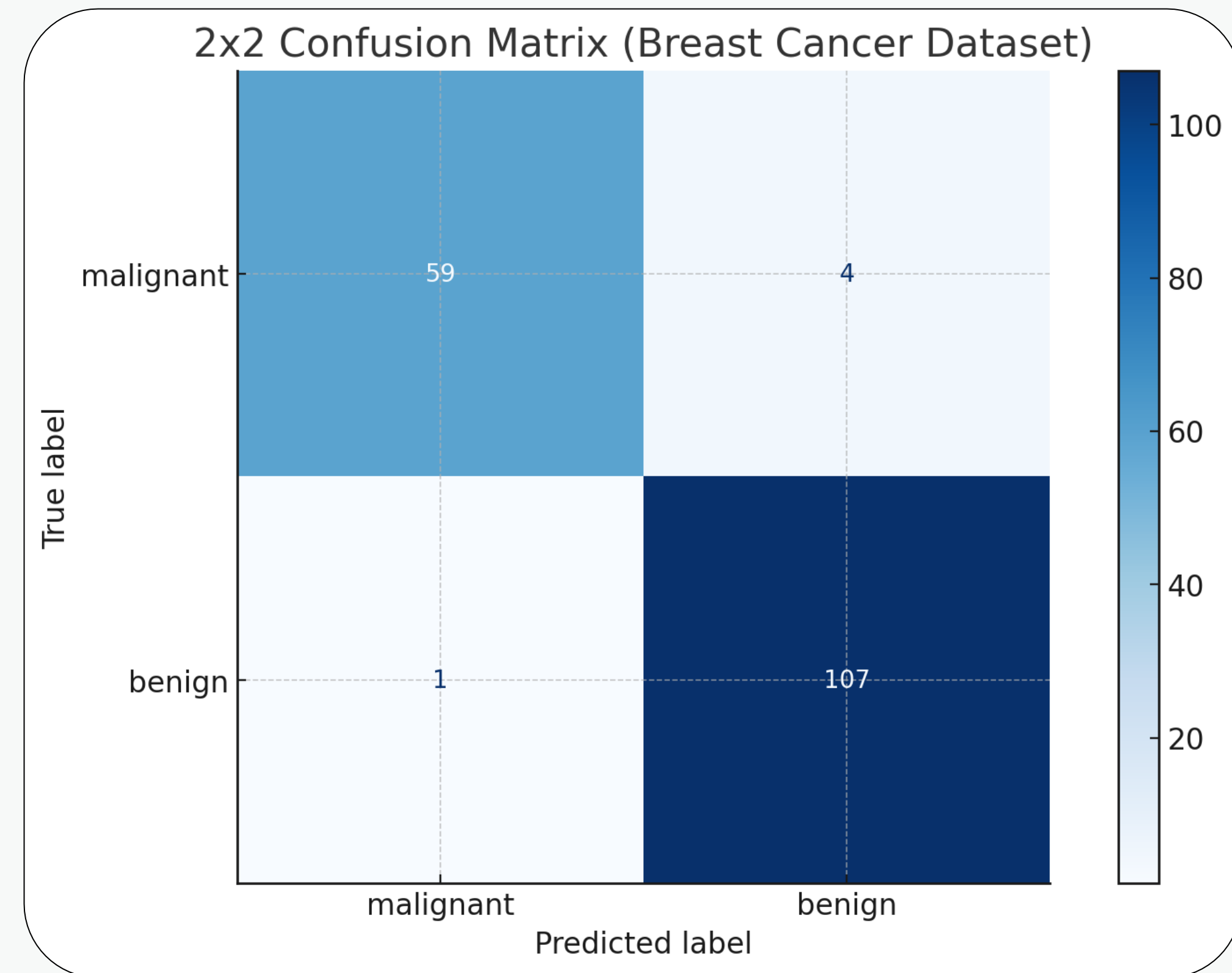
R^2 (결정 계수)

- y 의 전체 변동성 중 모델이 설명한 비율
- 범위: 0~1 이상 (1에 가까울수록 학습 성능 우수)

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}$$

분류 모델 평가 지표

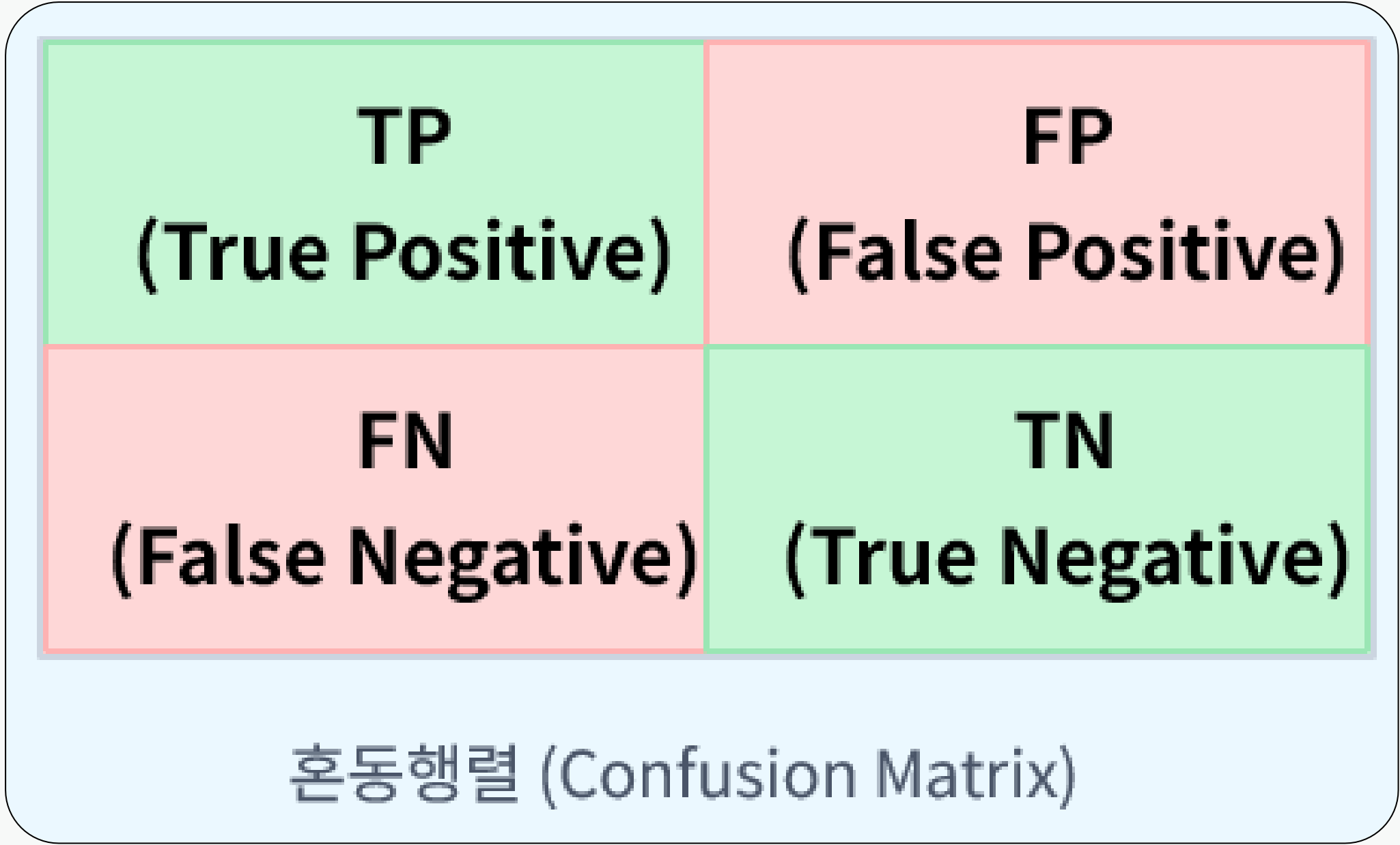
- Accuracy (정확도)
- Precision (정밀도)
- Recall (재현율)
- F1-Score



혼동 행렬이란?

- 예측 결과와 실제 값을 비교한 표

항목	설명
TP (True Positive)	실제 Positive를 모델이 Positive로 맞힘
TN (True Negative)	실제 Negative를 모델이 Negative로 맞힘
FP (False Positive)	실제 Negative인데 Positive로 잘못 예측
FN (False Negative)	실제 Positive인데 Negative로 잘못 예측



Accuracy (정확도)

- 전체 샘플 중에서 모델이 맞게 예측한 비율

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

정확도(Accuracy)의 한계

- 데이터가 불균형 할 때, Accuracy는 왜곡된 평가를 만듭니다.

예를 들어, 99%가 정상이고 1%만 이상인 데이터에서모델이 전부 “정상”이라고 예측해도 Accuracy는 99%입니다.

하지만 실제로는 이상 탐지 성능이 0인 셈이죠.

→ 이런 상황에서 Precision과 Recall이 필요합니다.

Precision (정밀도)

- 모델이 양성이라고 판단한 것 중 실제로 양성인 비율

$$\text{Precision} = \frac{TP}{TP + FP}$$

이메일 스팸 필터

- 일반 메일을 스팸으로 잘못 분류하면 안 됨
- Precision이 높아야 함

Recall (재현율)

- 실제 양성 중에서 모델이 맞게 양성으로 예측한 비율

$$\text{Recall} = \frac{TP}{TP + FN}$$

암 진단 모델

- 암 환자를 놓치면 안 됨
- Recall이 높아야 함

Precision vs Recall의 트레이드오프

- Precision $\uparrow$ 하면 Recall $\downarrow$ 될 가능성이 높음 (반대도 마찬가지)

예를 들어,스팸 필터 기준을 엄격히 $\rightarrow$ Precision 높아짐 (정상 메일은 거의 안 잘림)

하지만 스팸을 많이 놓침 $\rightarrow$ Recall 낮아짐

그래서 이 둘을 균형 있게 보려면 F1-score 사용

F1-Score

- Precision과 Recall의 조화 평균

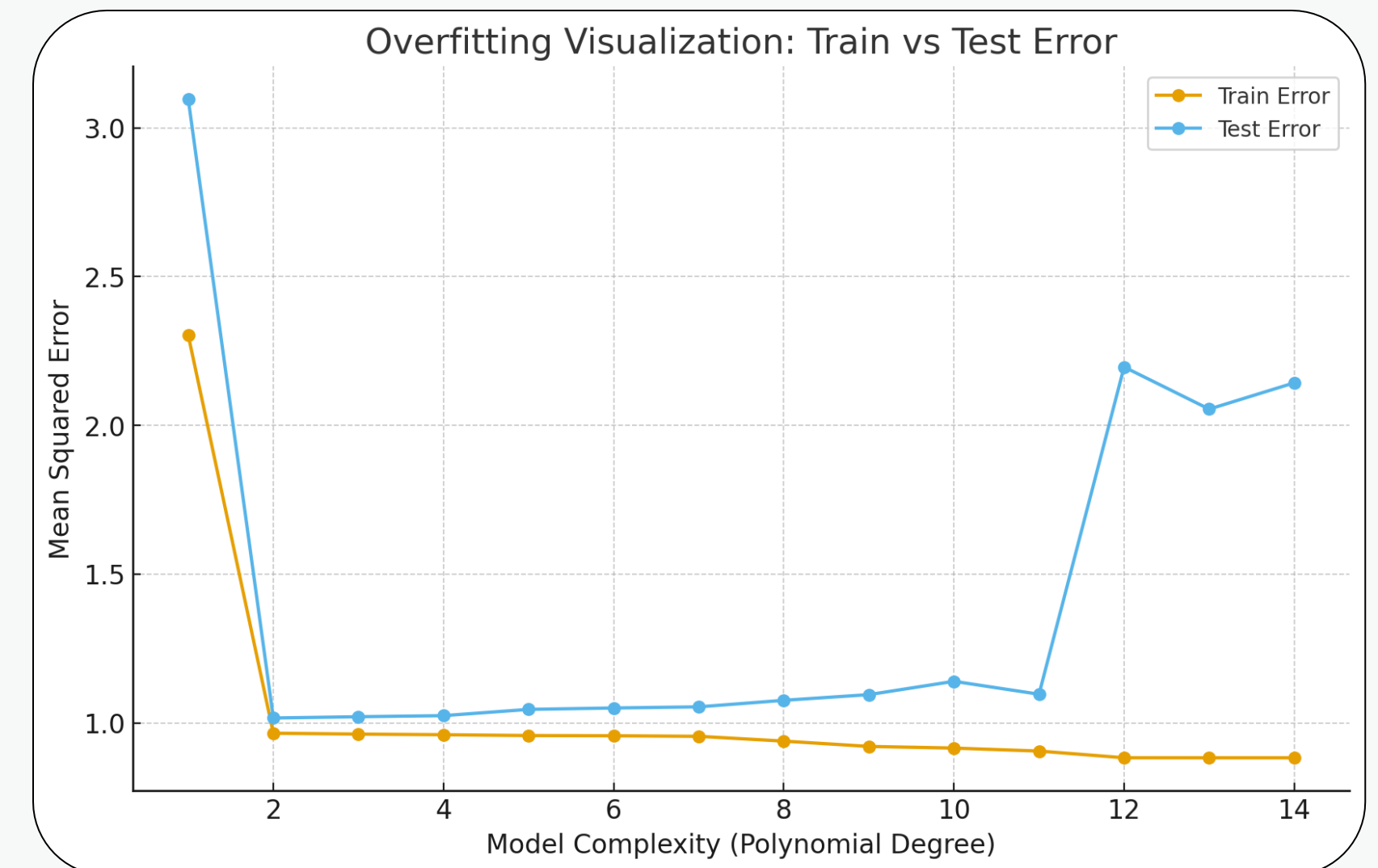
$$F1 = 2 \times \frac{Precision \cdot Recall}{Precision + Recall}$$

과적합

모델이 훈련 데이터에 지나치게 잘 적합되어,
새로운 데이터(테스트 데이터)에서는 성능이 떨어지는 현상을 의미합니다.

원인

너무 복잡한 모델 (예: 깊은 신경망, 큰 깊이의 결정트리)
훈련 데이터의 양이 적거나 품질이 낮을 때



교차 검증

- 모델을 보다 신뢰성 있게 평가하기 위해 데이터를 여러 번 나눠가며 학습/검증하는 기법
 - 모델의 일반화 성능을 정확히 평가 (과적합 해소)
-
- **K-Fold Cross Validation**
데이터를 K개의 폴드로 나누고, (K - 1)개로 훈련, 1개로 검증
K번 반복하고 평균 성능을 계산



하이퍼 파라미터 튜닝

최적의 하이퍼파라미터 값을 찾아 모델 성능 개선

- Grid Search

설정된 하이퍼파라미터의 모든 조합을 탐색계산 비용이 큼

- Random Search

임의의 조합을 일정 횟수 시도시간 대비 효율적



Scikit-learn



파이썬 머신러닝 라이브러리

데이터 분리

`train_test_split()` : 훈련용 / 평가용 데이터 분할

```
# 독립 변수(X): target 컬럼 제거
```

```
X = df.drop('target', axis=1)
```

```
# 종속 변수(y): target 컬럼만 사용
```

```
y = df['target']
```

```
# 데이터셋 분할: 80% 학습, 20% 테스트 / 레이블 비율 유지(stratify)
```

```
X_train, X_test, y_train, y_test = train_test_split(  
    X, y, test_size=0.2, random_state=42, stratify=y)
```

모델 훈련 및 평가를 위한 예측 수행

.fit() : 모델 훈련

.predict() : 훈련한 모델 기반으로 예측 값 생성

#모델 선택 및 훈련

```
model = RandomForestRegressor() #랜덤포레스트 회귀 예측 모델
```

```
model = XGBClassifier() #XGB 분류 예측 모델
```

```
model.fit(X_train, y_train)
```

#예측 수행

```
y_pred = model.predict(X_test)
```


모델 평가 - 회귀

실제 값과 예측 값을 토대로 지표 계산

```
# 개별 지표 계산
```

```
mae = mean_absolute_error(y_true, y_pred)
```

```
mse = mean_squared_error(y_true, y_pred)
```

```
rmse = np.sqrt(mse)
```

```
r2 = r2_score(y_true, y_pred)
```

모델 평가 - 분류

실제 값과 예측 값을 토대로 지표 계산

```
# 개별 지표 계산
```

```
accuracy = accuracy_score(y_test, y_pred)
```

```
precision = precision_score(y_test, y_pred)
```

```
recall = recall_score(y_test, y_pred)
```

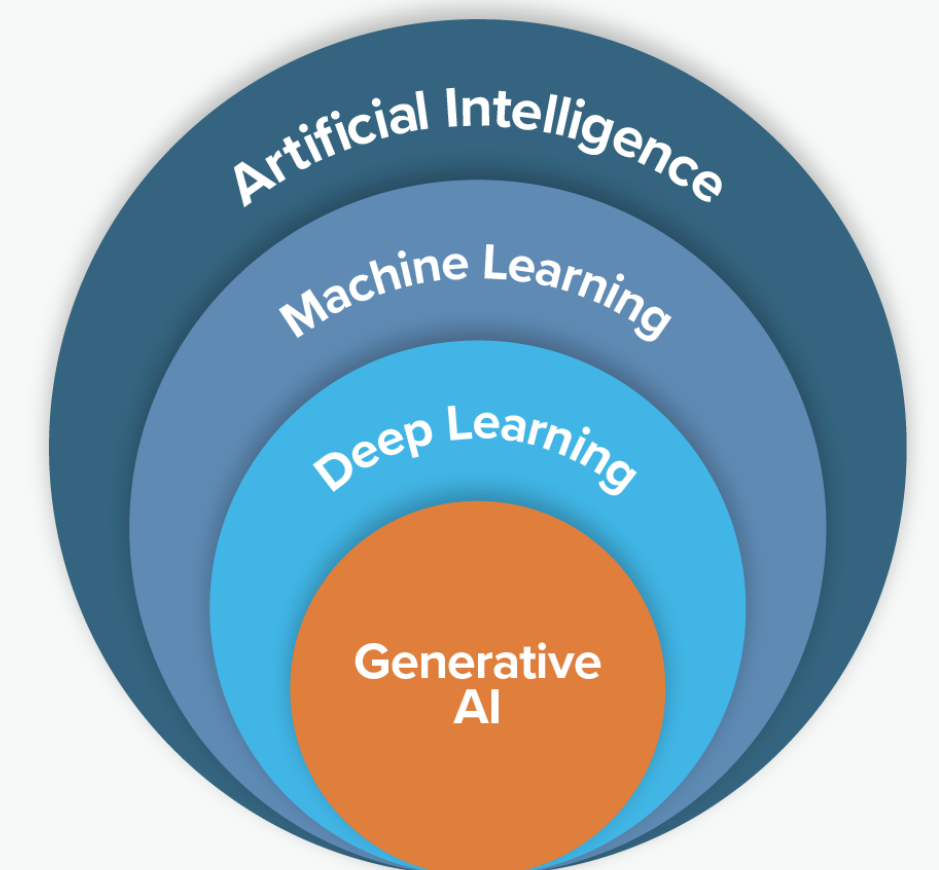
```
f1 = f1_score(y_test, y_pred)
```

딥러닝



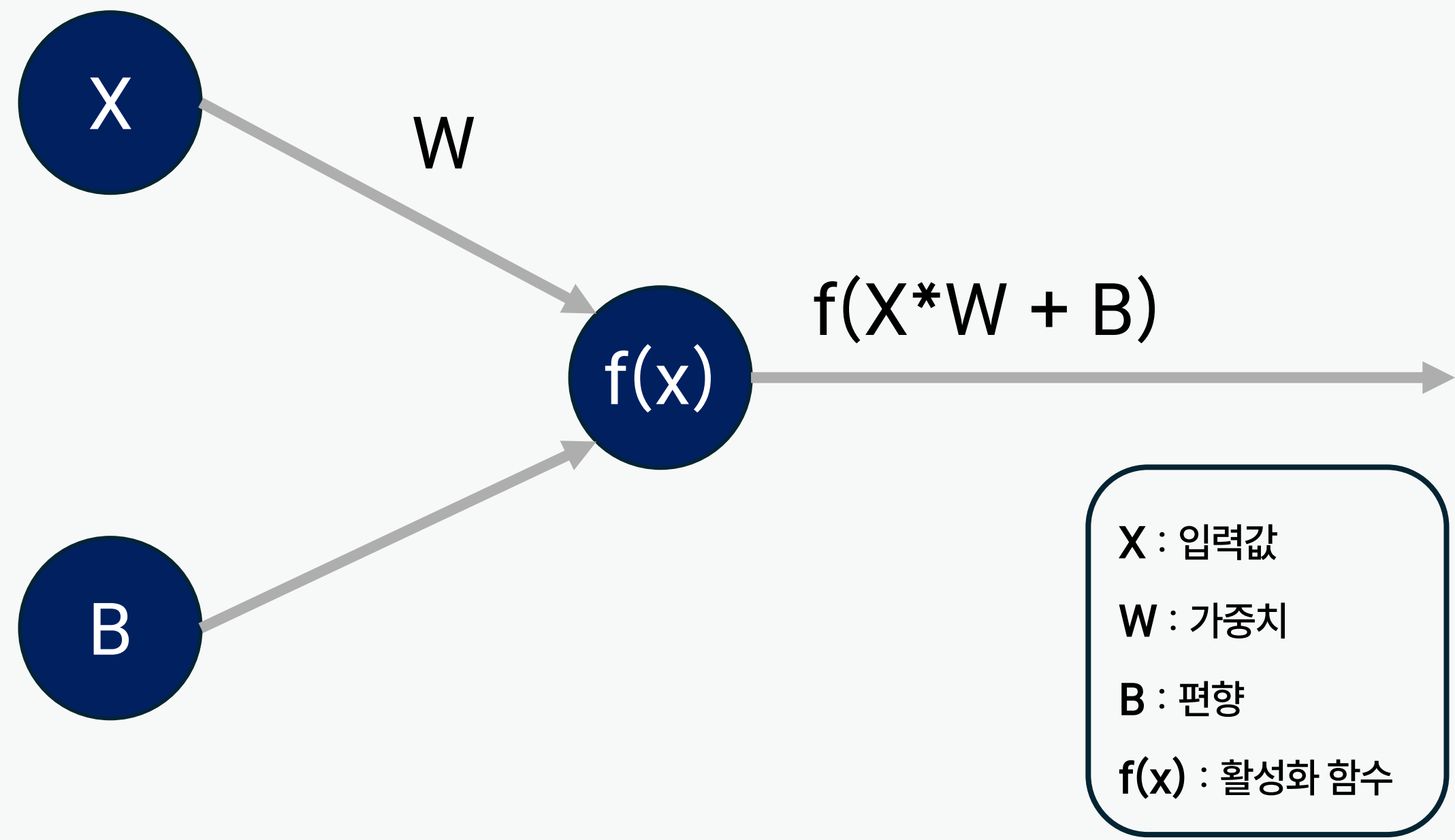
딥러닝이란?

인공신경망을 기반으로 데이터를 학습하여 복잡한 문제를 해결하는 기술



퍼셉트론

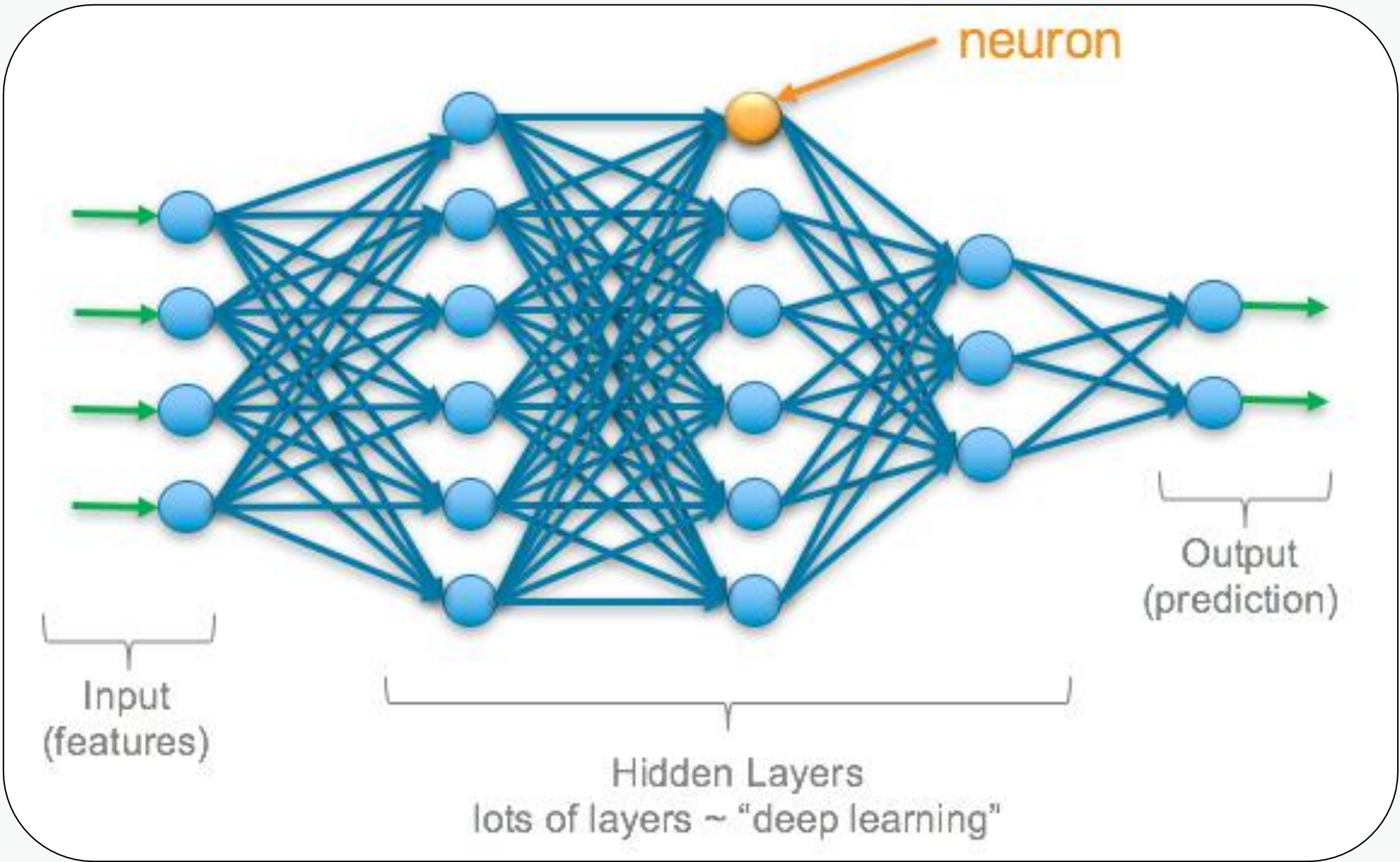
인공 신경망의 가장 기본적인 단위 모델



구성 요소	설명
입력(Input)	여러 개의 입력값
가중치(Weight)	각 입력에 곱해지는 값
편향(Bias)	임계값을 조절하는 추가 입력
활성화 함수	결과값을 출력(0 또는 1)으로 결정하는 함수

심층 신경망

여러 개의 은닉층(hidden layer)을 포함한 신경망 구조



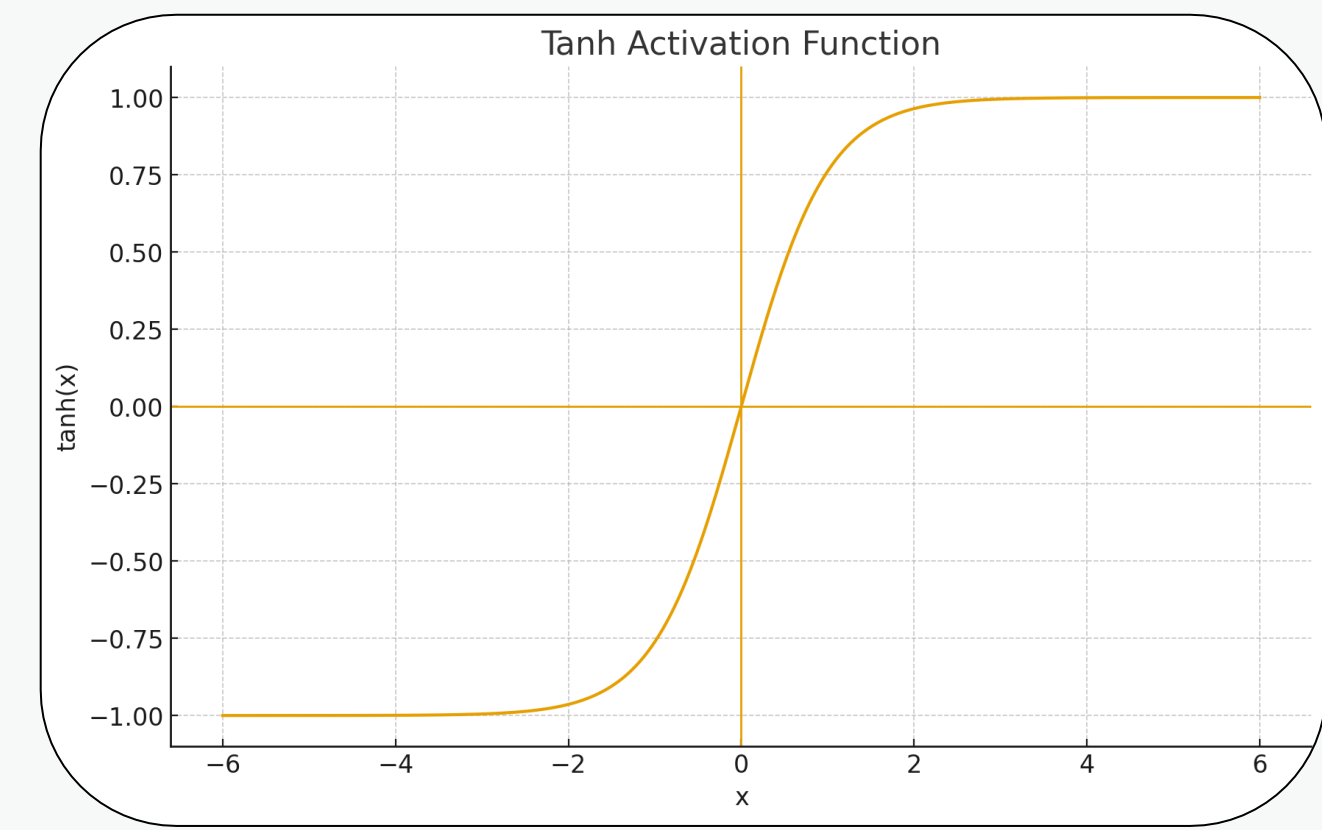
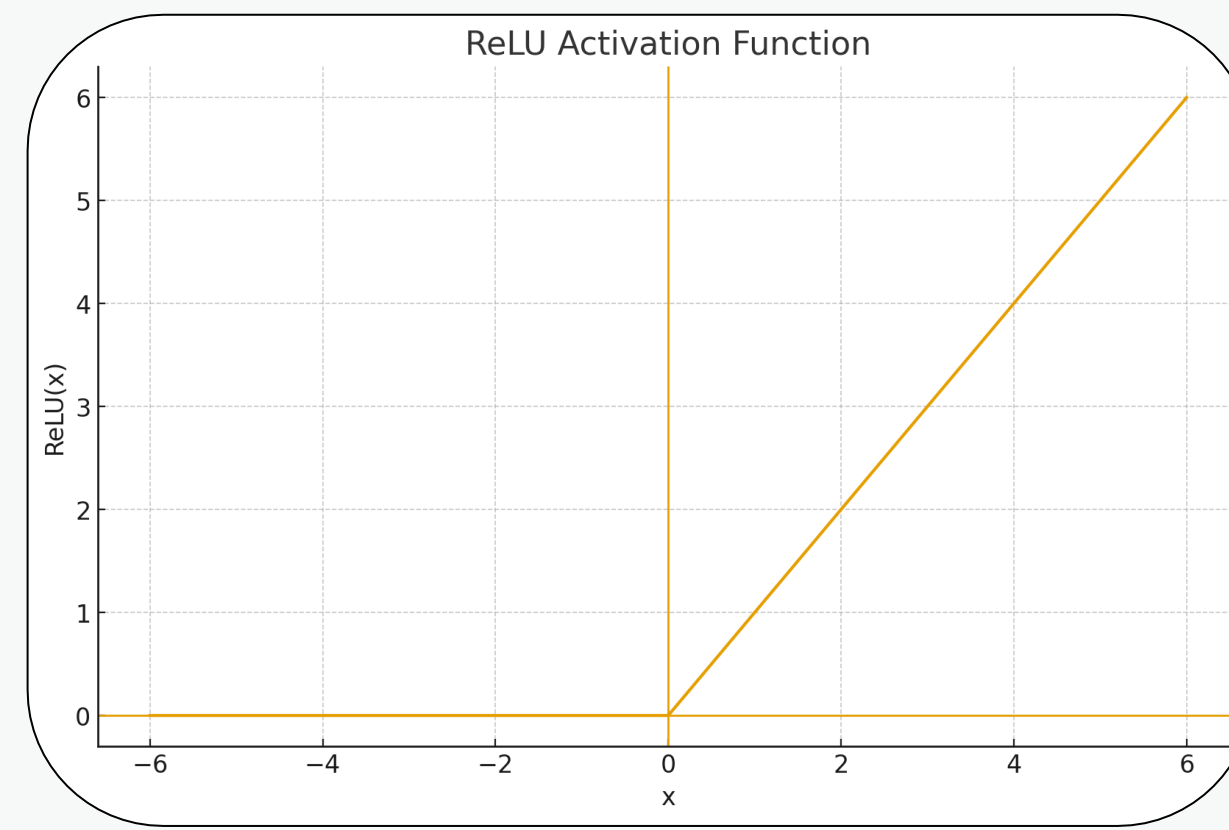
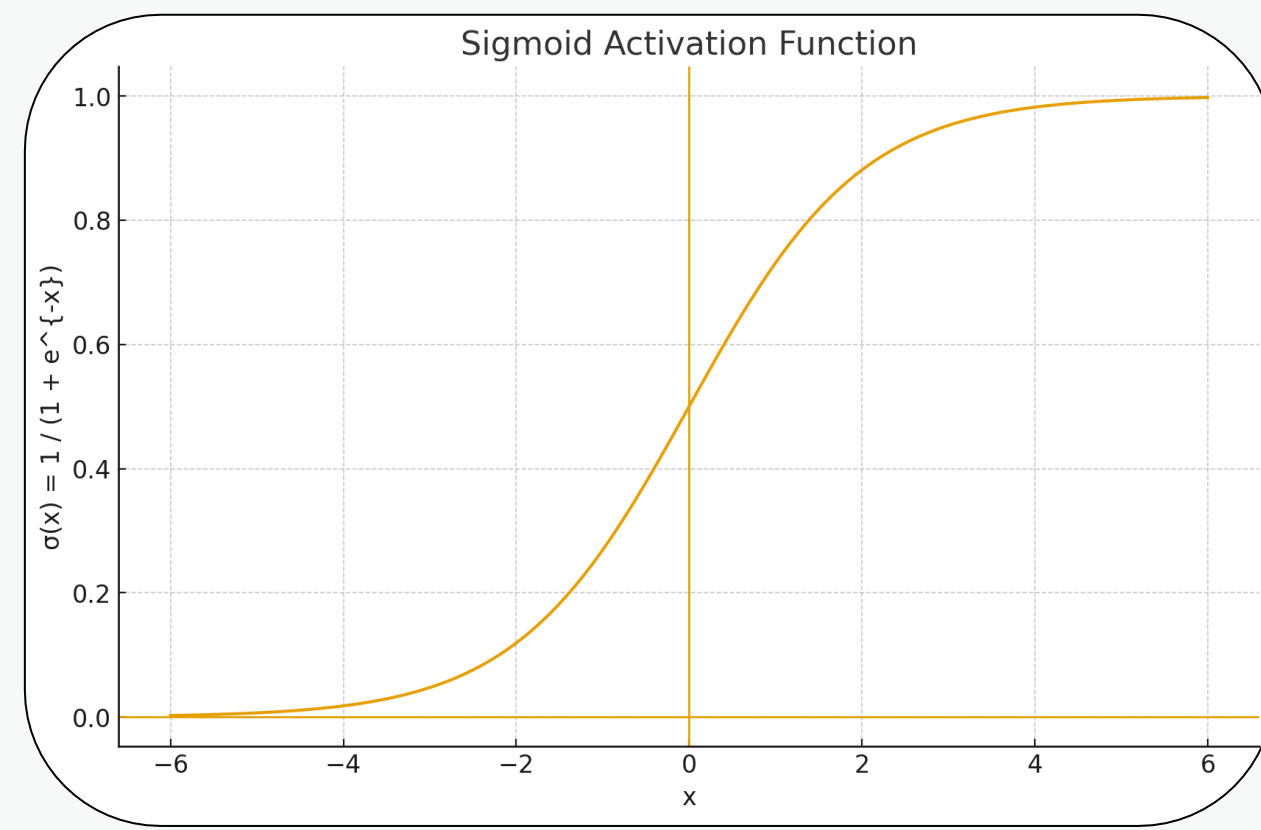
종류	주요 역할	구성 요소
입력층	원본 데이터를 신경망에 전달	피쳐 수만큼 뉴런 구성
히든층	특징 추출 및 학습, 패턴 인식	가중치, 편향, 활성화 함수
출력층	최종 예측 결과 출력	문제에 따른 뉴런 및 활성화 함수

활성화 함수

뉴런(노드)의 입력 값에 비선형 변환을 적용하여 출력 값을 생성하는 함수.

- 목적

딥러닝 모델이 비선형 문제를 학습하고 복잡한 패턴을 표현할 수 있게 함.



 손실 함수

모델의 예측 값과 실제 값 간의 차이를 수치로 표현한 함수.

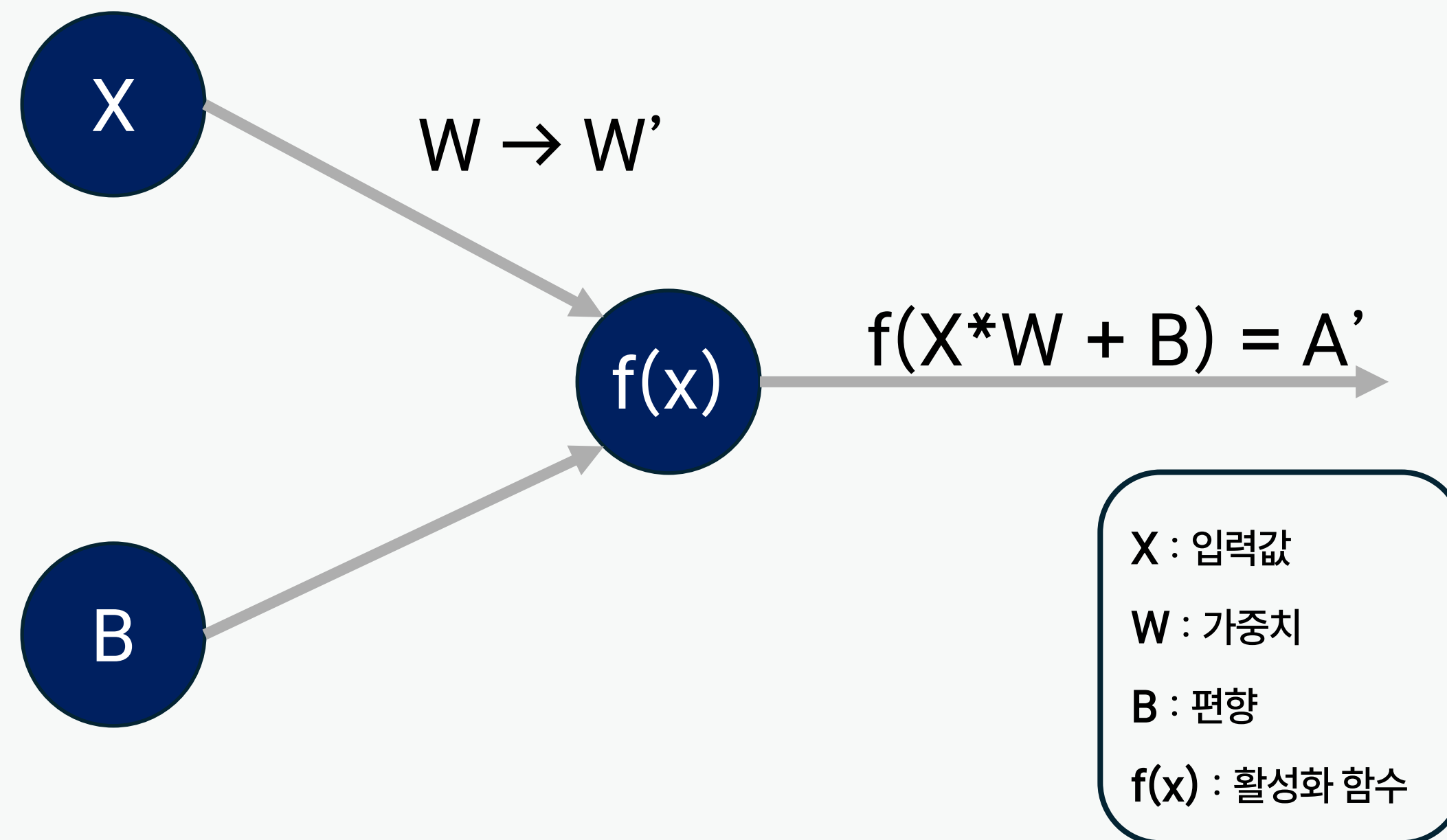
- 목적

모델을 학습할 때, 이 손실 값을 최소화하는 방향으로 가중치를 업데이트함.

문제 유형	손실 함수
회귀	MSE, MAE
이진 분류	Binary Crossentropy
다중 분류	Categorical Crossentropy

■ 딥러닝 최적화 원리

- 예측 값과 실제 값의 오차(손실 함수 값)을 최소화하는 방향으로 학습
- 가중치를 조정하면서 학습 진행



출력 값 A' 와 실제 값 A 의 오차가 작아지는 방향인
가중치 W 를 W' 로 변경하여 학습 진행

경사하강법 & 역전파 알고리즘을 사용하여 학습 진행

경사하강법

손실 함수의 기울기를 따라 가중치를 업데이트하여 손실을 최소화하는 최적화 알고리즘.

$$\theta = \theta - \alpha \nabla J(\theta)$$

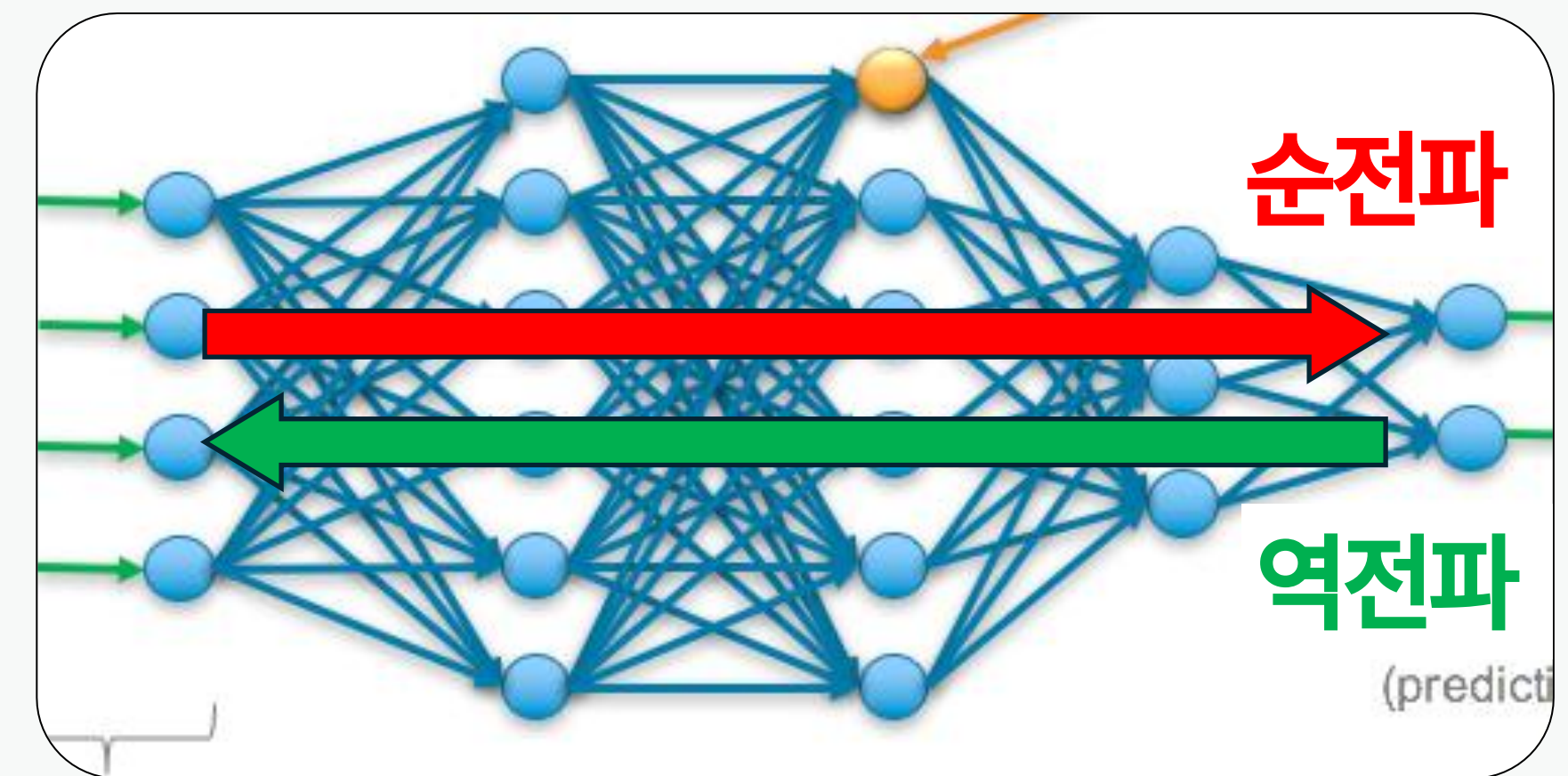
- θ (세타): 모델의 파라미터(가중치)
- α (알파): 학습률(Learning Rate) → 한 번에 얼마나 이동할지 결정
- $\nabla J(\theta)$: 손실 함수 $J(\theta)$ 의 기울기(Gradient)

역전파

출력층에서 계산된 오차를 입력층 쪽으로 거슬러 전파시키며
각 가중치의 기울기를 계산하는 알고리즘.

- 동작 방식

1. 순전파(Forward pass): 입력 → 출력 예측
2. 손실 계산
3. 역전파: 손실에 대한 가중치 기여도 계산
4. 그라디언트(기울기)로 가중치 업데이트



과적합 방지 기법

1. 드롭 아웃

학습 시 무작위로 일부 뉴런과 연결을 제거하여 과적합을 방지하는 기법.

- 특징

학습 중 특정 비율(예: 0.5)만 활성화.

테스트 시에는 모든 뉴런 사용하지만 가중치는 학습 시에 맞게 조정.

2. L1/L2 정규화

모델 가중치에 대한 제한(패널티)을 추가하여 과적합을 방지하는 기법.

Tensorflow



TensorFlow

파이썬 딥러닝 라이브러리

모델 정의

`sequential()` : 모델 생성

`.add()` : 레이어 생성

```
# Sequential 모델 생성
model = Sequential()

# 첫 번째 은닉층: 64개 노드, 입력 크기 지정
model.add(Dense(64, activation='relu', input_shape=(X_train.shape[1],), name='dense_1'))

# 두 번째 은닉층
model.add(Dense(32, activation='relu', name='dense_2'))

# 출력층: 이진 분류이므로 sigmoid 활성화 함수 사용
model.add(Dense(1, activation='sigmoid', name='output'))
```


모델 컴파일

.compile(): 모델 컴파일

```
model.compile(  
    optimizer='adam',  
    loss='binary_crossentropy',  
    metrics=['accuracy']  
)
```

Adam 옵티마이저 사용
이진 분류 손실함수
성능 평가 지표: 정확도

optimizer : 가중치를 조절하는 방법 설정

loss : 모델이 얼마나 틀렸는지 계산하는 기준 설정

metrics : 모델 평가 지표 설정

모델 훈련 및 평가를 위한 예측 수행

`.fit()` : 모델 훈련

`.predict()` : 훈련한 모델 기반으로 예측 값 생성

```
history = model.fit(  
    X_train, y_train,  
    validation_split=0.2,      # 학습 데이터 중 20%는 검증용  
    epochs=50,                # 학습 반복 횟수  
    batch_size=32,            # 배치 크기  
    verbose=1                  # 학습 과정 출력  
)  
  
y_pred = model.predict(X_test)
```



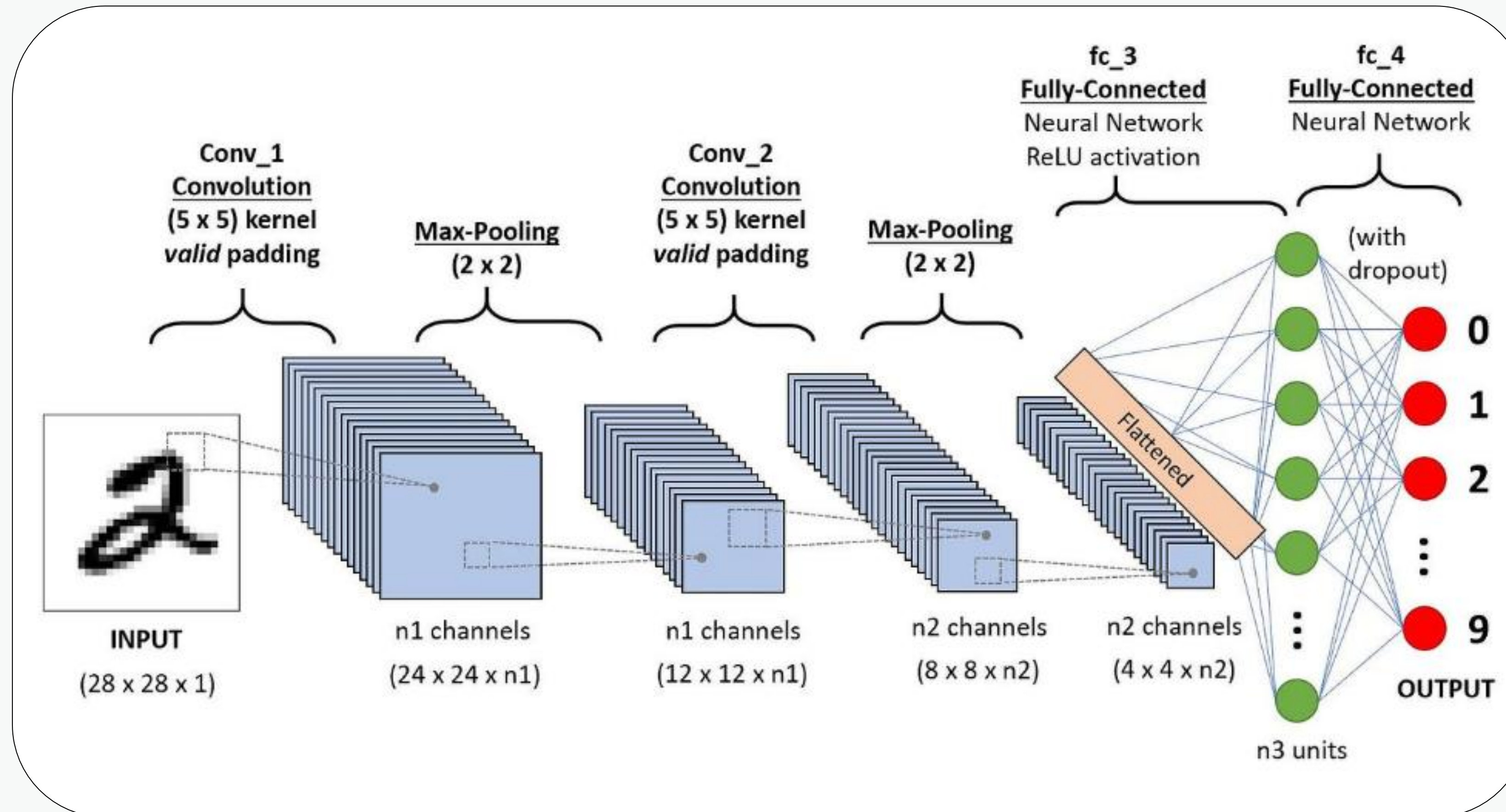
머신러닝 vs 딥러닝

구분	머신러닝 (Machine Learning)	딥러닝 (Deep Learning)
데이터 요구량	비교적 적은 데이터로도 학습 가능	대량의 데이터 필요 (수천~수백만 개)
모델 구조	선형회귀, SVM, 결정트리 등 비교적 단순한 모델	CNN, RNN 등 깊은 계층 구조의 신경망 사용
학습 시간	빠름 (보통 CPU로도 충분)	느림 (고성능 GPU/TPU 필요)
성능	단순 문제나 적은 데이터에서 우수	이미지, 음성, 자연어 처리 등 복잡한 문제에서 월등
대표 프레임워크	Scikit-Learn, XGBoost 등	TensorFlow, PyTorch 등
적용 분야	가격 예측, 단순 분류, 의사결정 지원 등	자율주행, 음성인식, 이미지 분류, 챗봇 등

CNN(Convolutional Neural Network)

이미지나 영상과 같은 공간적 패턴을 인식하기 위해
합성곱 연산을 통해 특징을 추출하는 딥러닝 모델

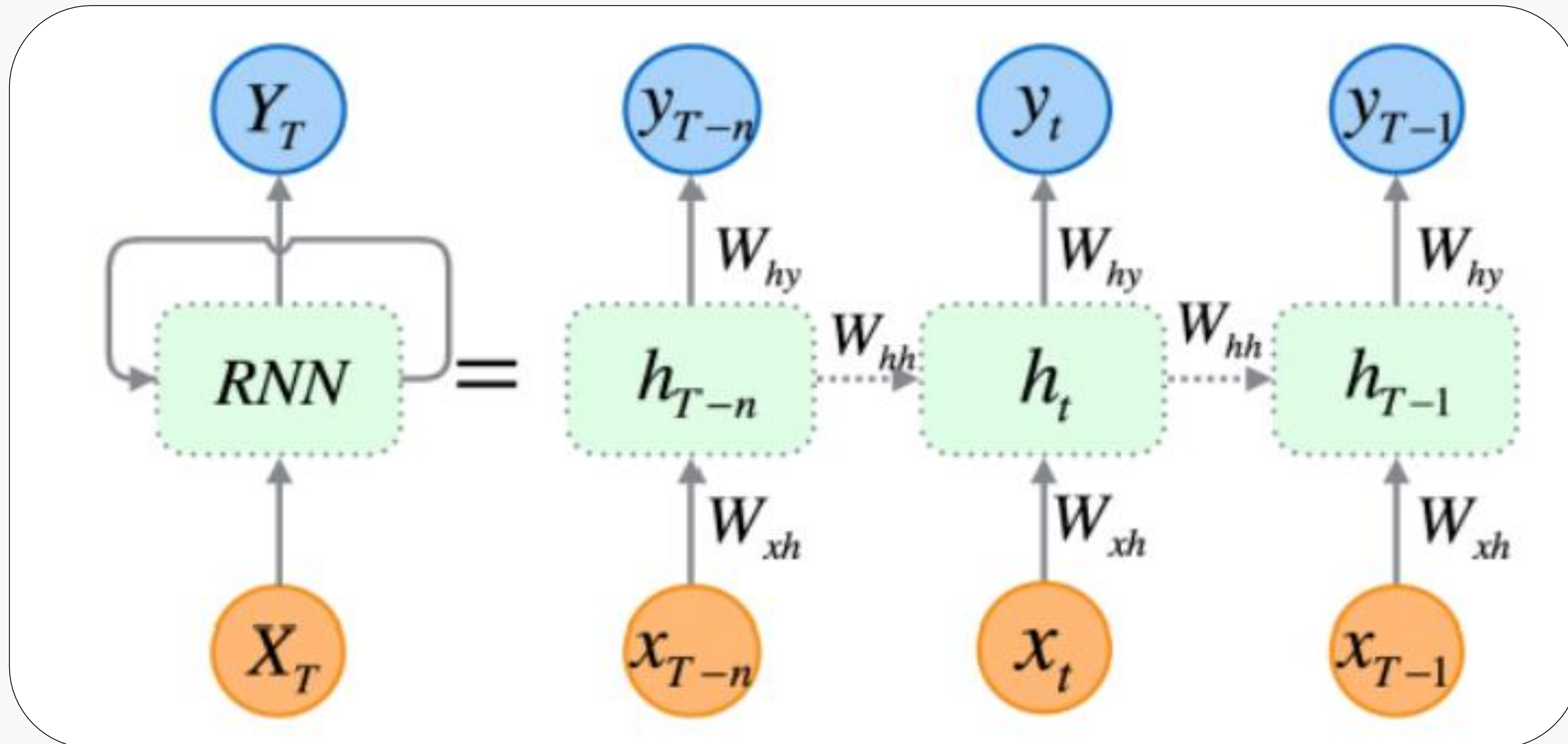
CNN(Convolutional Neural Network)



RNN(Recurrent Neural Network)

시계열 데이터나 문장처럼 순서가 있는 데이터를 처리하기 위해
이전 상태의 정보를 순환적으로 기억하며 학습하는 딥러닝 모델

❖ RNN(Recurrent Neural Network)



언어모델





언어 모델

자연어 데이터를 학습해 텍스트 생성, 요약, 번역 등
다양한 언어 처리 작업을 수행할 수 있는 AI 모델



언어 모델 발전 - 1

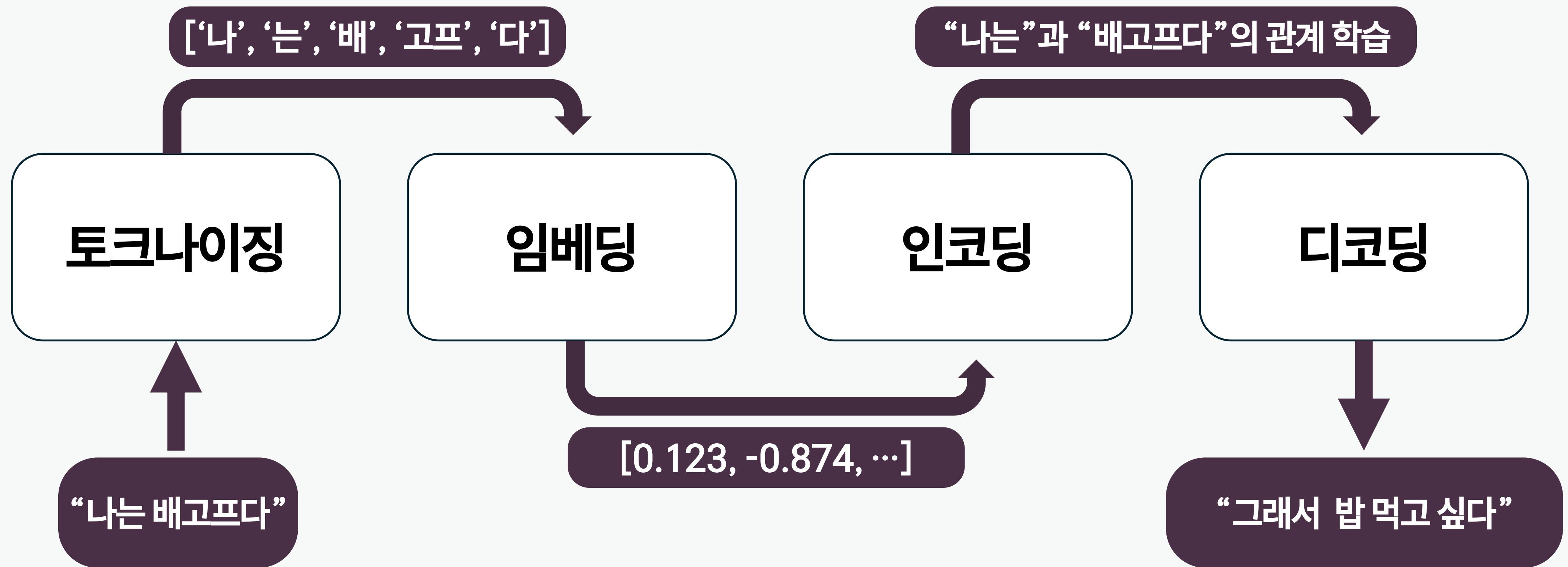
모델	단점	다음 모델	핵심 변화	개선 포인트
DNN	단어 순서 이해 X	RNN	단어를 시간 순서대로 처리	문장의 흐름을 따라갈 수 있도록
RNN	앞부분 내용 잊어버림	LSTM	기억장치 추가	중요한 정보 오래 기억
LSTM	아주 긴 문장 처리 어려움	Seq2Seq 기반 LSTM	인코더에서 이해 디코더에서 생성	문장을 구조적으로 요약
Seq2Seq 기반 LSTM	문장 전체를 하나로 압축하면 정보 누락	Attention + Seq2Seq	문장 내 중요한 부분에 집중	중요한 단어에 더 집중



언어 모델 발전 - 2

모델	단점	다음 모델	다음 모델의 핵심 변화	개선 포인트
Attention + Seq2Seq	- 여전히 느린 순차 처리	Transformer	병렬 처리 가능	한꺼번에 처리해서 빠르게!
Transformer	- 구조 복잡 + 자원 많이 필요	GPT-1	디코더만 사용 → 생성 특화	생성 중심으로 간단하게
GPT-1	- 모델이 작아 똑똑하지 않음	GPT-2	파라미터 수 15억 → 생성력 고도화	모델 크기 키워서 능력 확장
GPT-2	- 가짜텍스트 생성 논란 - 여전히 작업별 학습 필요	GPT-3	1750억 파라미터 + Few-shot	엄청나게 커져서 다양한 작업 자동처리
GPT-3	- 너무 크고 비용 많이 듦 - 사실과 다른 말도 함	GPT-3.5	휴먼 피드백(RLHF)로 대화 능력 향상	인간처럼 대화하고 안전하게

언어 모델 프로세스



토크나이징(토큰화)

긴 문장을 모델이 다루기 쉬운 **작은 조각(토큰)** 으로 자름.
적당한 크기로 잘라야 다음 단계에서 의미를 붙이기 쉬워짐.

- 작동 원리

단어 단위: “나는”, “밥을”, “먹는다”

서브워드(자주 쓰는 덩어리) 단위: “먹”, “는다”

- 예시

문장: “나는 오늘 진짜 배고파.”

토큰: [“나”, “는”, “오늘”, “진짜”, “배”, “고파”, “.”]

어디서 자를지는 토크나이저에 따라 자동으로 결정

임베딩(벡터화)

토큰을 의미를 담은 **숫자 벡터로 변환**.

숫자로 바뀌야 모델이 계산할 수 있고, 비슷한 단어를 가깝게 배치할 수 있음.

- 작동 원리

토큰 → 고정 길이 벡터(예: 768차원, 1024차원 등)

비슷한 의미를 가진 단어는 비슷한 숫자 패턴을 가짐

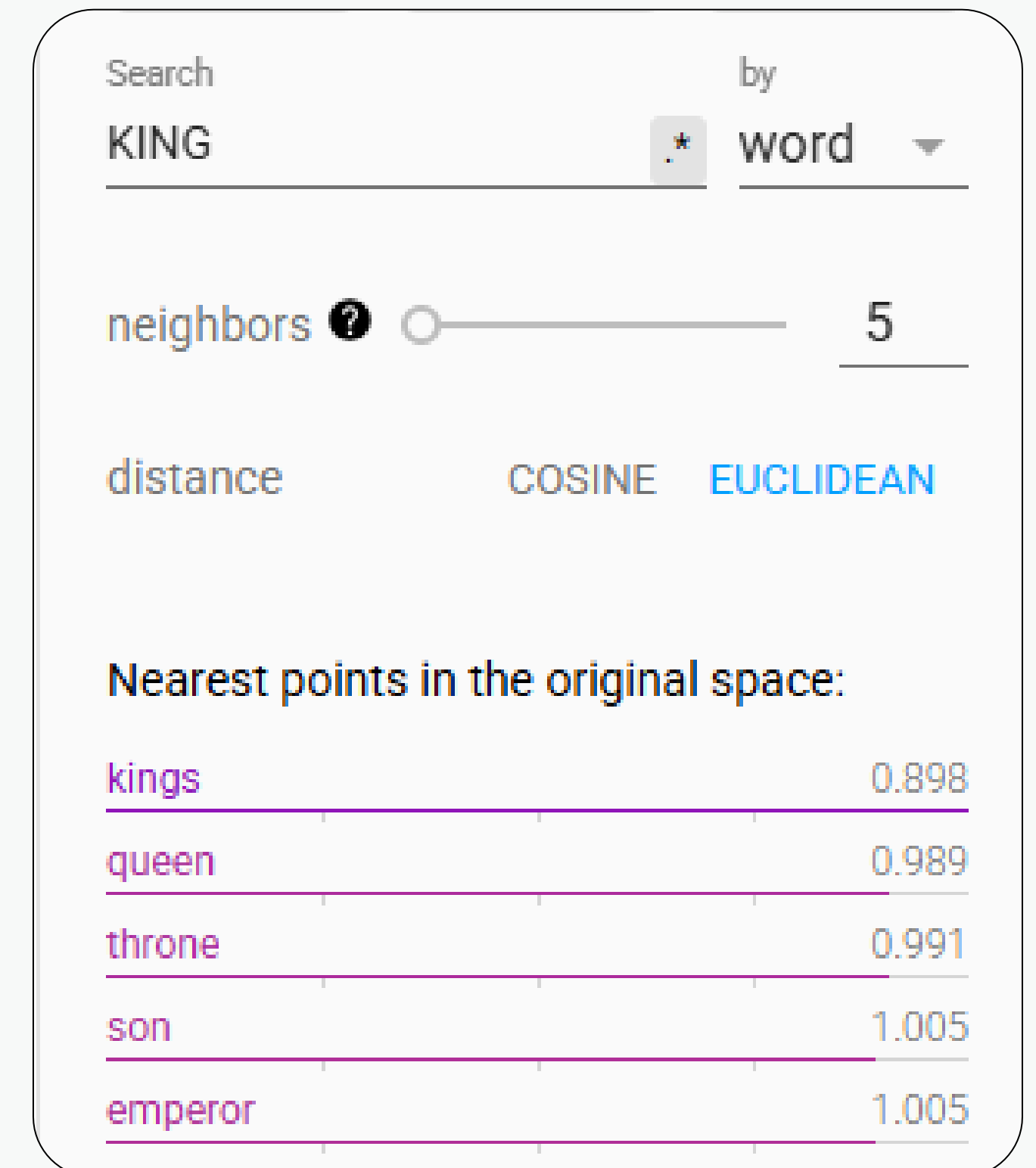
예: “밥”과 “식사”는 서로 가까운 벡터 값

예: “바다”와 “하늘”도 어느 정도 가까움

- 예시

토큰: “밥”

임베딩 결과 → [0.12, -0.88, 0.31, 0.02, -0.54, ...] (768개의 숫자)



Search by

neighbors

distance

Nearest points in the original space:

kings	0.898
queen	0.989
throne	0.991
son	1.005
emperor	1.005



인코딩

문장 속에서 단어들이 서로 어떤 관계를 갖는지 이해하는 단계.

- 작동 원리

모든 토큰끼리 서로를 바라보며 관계 계산 (Self-Attention)

중요한 단어나 연결된 단어에 더 집중

예를 들어 “배고파”는 “진짜”와 세게 연결됨, “나는”과는 적당히 연결됨

- 예시

문장: [“나”, “는”, “오늘”, “진짜”, “배”, “고파”]

단어	가장 관련 있는 단어	중요도
고파	배	0.9
고파	진짜	0.7
오늘	진짜	0.4
나는	고파	0.2

디코딩

인코딩 결과를 바탕으로 다음 단어를 하나씩 예측하며 문장을 생성하는 단계.

- 작동 원리

“다음에 올 단어들의 후보” 중에서 가장 자연스러운 단어 선택
이전 단어 정보를 참고해서 하나씩 이어붙이며 생성
예: “나는 오늘 배고파” → 다음 단어 “그래서”

- 예시

입력: “나는 오늘 진짜 배고파”

단계	생성된 단어	문장
1	그래서	그래서
2	밥을	그래서 밥을
3	먹고	그래서 밥을 먹고
4	싶다	그래서 밥을 먹고 싶다

GPT에 인코더가 없는데?

“인코더 모듈”이 없다는 뜻이지,
입력을 이해하는 과정(인코딩 성격) 자체가 없는 건 아님.

디코더 내부의 **Self-Attention**이 입력 토큰들 사이 관계를 계산하면서
사실상 인코딩 역할까지 겸함.

RNN / LSTM (Seq2Seq 이전)

단어(토큰)를 하나씩 순서대로 읽으며, 이전 상태를 기억해 다음 단어 판단
“나는” → “밥을” → “먹고” → “싶다” 처리하며 일부 문맥 정보 유지

- 장점

문장 전체를 읽을 수 있게 됨 (순차적으로)

이전 모델 문제	Seq2Seq
짧은 문맥만 처리	긴 문장도 입력 가능
입력을 이해해도 출력 생성 능력은 부족	입력 인코딩 + 출력 디코딩으로 번역/요약 가능
"문장 전체 변환" 불가능	입력 → 출력 구조 최초 도입

Seq2Seq

입력 문장을 이해하고 → 출력 문장을 생성하는 구조
대표적으로 번역, 요약 같은 태스크에서 사용됨.

“입력 → 인코더 → 디코더 → 출력”이라는 고전적인 구조가 여기서 탄생.

- 작동 원리

인코더가 입력 문장을 순서대로 읽고 하나의 벡터로 요약

예: “나는 배고프다” → 하나의 문장 요약 벡터 (context vector)

디코더가 그 요약 벡터를 보고 출력 문장을 생성

예: → “I am hungry” (단어 하나씩 생성)

→ 이 구조 덕에 입력과 출력이 다른 시퀀스 (문장, 언어 등)를 다룰 수 있게 됨

Attention

입력의 각 단어들 중에서 “중요한 부분에 더 집중”할 수 있게 만든 기술

- 작동 원리

디코더가 문장을 생성할 때 입력 문장의 전체 단어를 다시 바라보며,
각 단어의 중요도(가중치)를 계산해 반영함

예: “나는 진짜 너무 배고파”

“배고파” → 디코더는 이 단어에 가장 집중

“진짜”, “오늘” → 덜 집중

생성 중인 단어	입력 문장 중 주로 집중한 단어
“그래서”	“배고파” (0.9), “진짜” (0.6)
“밥을”	“배” (0.8), “고파” (0.7)
“먹고”	“밥” (0.8), “오늘” (0.3)

Transformer

문장 내 모든 단어를 동시에 비교해서 이해하고 생성할 수 있는 혁신적인 모델 구조.
“앞에서부터 차례로” 처리하지 않아도 되어 빠르고 정확한 언어 처리가 가능해짐.

- 작동 원리

1. Self-Attention을 통해 문장 전체 단어의 관계를 한 번에 계산

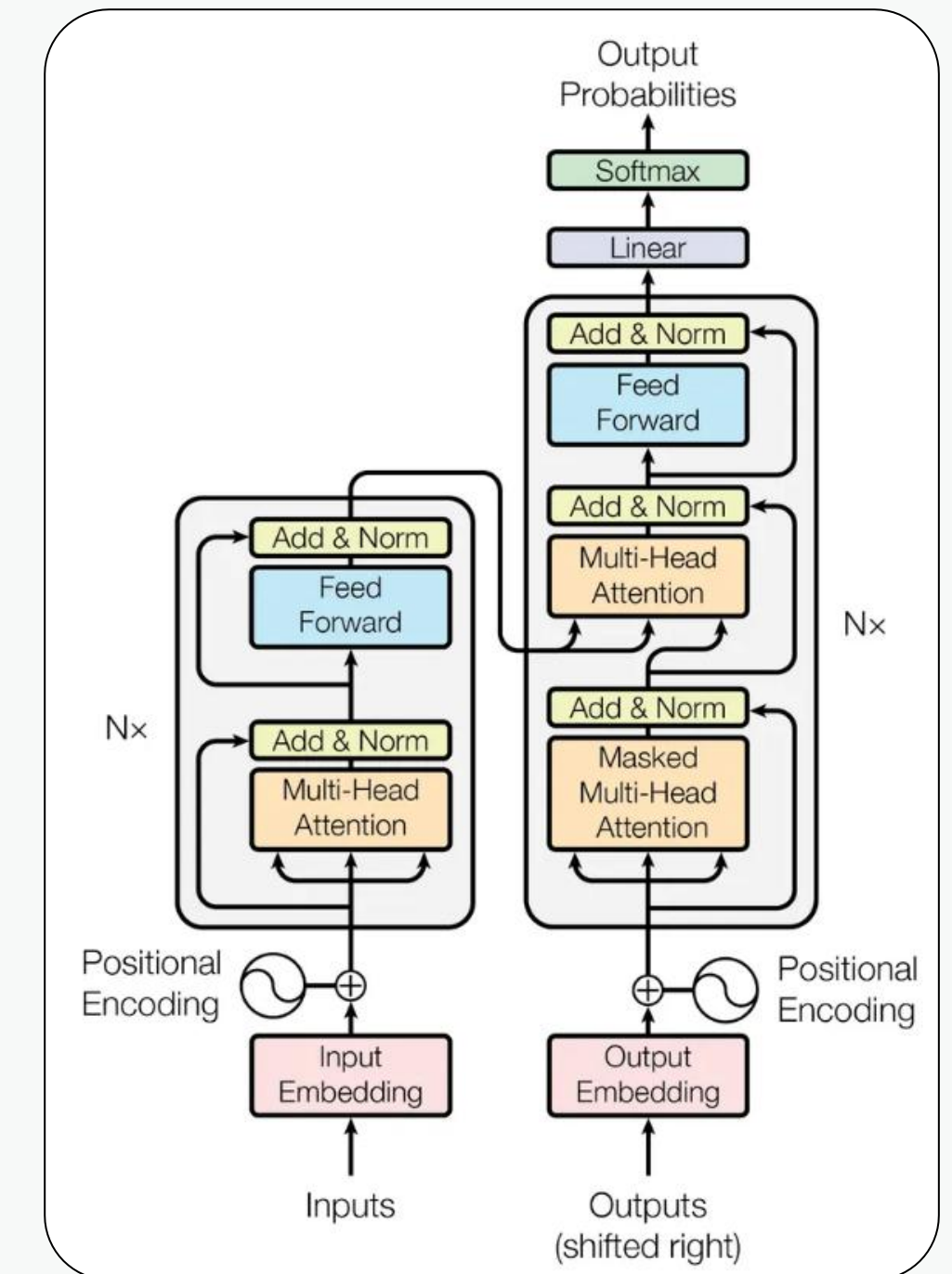
예: “나는 오늘 진짜 배고파” → “배고파”는 “진짜”와 관계가 강함, “오늘”과는 약함

2. 인코더와 디코더로 구성 (둘 다 Attention 사용)

인코더: 입력 문장 전체를 이해 (예: “나는 오늘 진짜 배고파”의 의미 벡터 생성)

디코더: 이해한 내용을 바탕으로 새 문장 생성 (예: “I’m so hungry today”)

3. 병렬 처리 가능





Self-Attention

문장 안의 단어들끼리 서로 연결되며 의미를 파악하는 기술
문장 안에서 각 단어가 같은 문장 안의 다른 단어들과 관련성을 계산

- 작동 방식
문장 : “나는 오늘 진짜 배고파”

기준 단어	연결되는 단어	중요도
고파	배	0.9
고파	진짜	0.8
고파	오늘	0.3

GPT(Generative Pre-trained Transformer)

트랜스포머 모델의 "디코더 부분만 사용"해서, 다음에 올 단어를 예상하며 문장을 생성하는 언어모델.

- 작동 원리

1. 입력 문장을 토큰나이징, 임베딩한 뒤 디코더에 입력
2. 디코더 내부의 셀프 어텐션으로 문맥을 이해
 - 단어 사이의 연결성을 보고 “무슨 뜻인지” 파악
3. 이해한 내용을 기반으로 다음 단어를 하나씩 생성
 - “나는 오늘 배고파” → “그래서” → “밥을” → “먹고 싶다”

Huggingface



HUGGING FACE

다양한 AI 모델과 데이터셋을 쉽게 공유하고 활용할 수 있는 오픈소스 플랫폼

Huggingface-pipeline

NLP, 컴퓨터 비전, 음성 등 다양한 오픈소스 AI 모델을 간단하게 사용할 수 있는 기능



언어모델 파라미터

파라미터	설명	값의 범위	특징
temperature	출력 단어 확률 분포를 조절	0.0 ~ 2.0 (일반적으로 0.7 기본)	낮을수록 결정적이고 일관적인 답변, 높을수록 창의적이고 다양한 답변
top-p	누적 확률이 p 이하인 단어 후보만 사용	0.0 ~ 1.0 (보통 0.8~0.95)	확률 기반으로 유동적으로 후보군 조정 → 다양성과 품질 균형
top-k	상위 k개의 단어 후보만 고려	정수 값 (보통 40 또는 50)	확률 높은 단어만 선택 → 무작위성 감소, 응답 안정성
max_tokens	생성할 최대 토큰 수	모델/사용 목적에 따라 설정 (예: 256, 512, 2048)	생성 길이 제한, 비용 절약 및 과도한 생성 방지

Temperature

단어 선택의 자유도"를 조절

- 원리

단어의 확률 분포 곡선을 조절낮을수록
"제일 확률 높은 단어"에 더 집중높을수록
"낮은 확률의 단어"도 선택 가능하게 함

- 예시

문장 완성하기: "하늘은 _ _ _ _ 다."

temperature=0.2 (보수적):-> "파랗다" (가장 적절한 정답)

temperature=1.0 (자유롭게):-> "파랗다", "맑다", "아름답다", "높다" 다양하게 나올 수 있음

temperature=1.5 (완전 창의적):-> "꿈같다", "춤춘다", "감정을 비춘다" ... 엉뚱하지만 창의적

Top-k

상위 k개만 남기고 나머지는 버리기

- 원리

후보 단어들을 확률 순으로 정렬 → 상위 k개만 고려

- 예시

위의 후보 단어 중

top-k=2 → 파랗다, 맑다만 후보

top-k=5 → 파랗다, 맑다, 흐리다, 높다, 아름답다까지 포함"

단어 후보	확률
파랗다	40%
맑다	30%
흐리다	10%
높다	5%
아름답다	4%
기타	11%

Top-p

확률이 높은 단어 중 누적 % 안에 드는 후보만 보고 선택

- 원리

100개 단어 중 확률이 높은 몇 개로 압축

예: 전체 단어 중 누적 확률이 0.9(90%)에 해당하는 후보만 고려

- 예시

하늘은 _ _ _ _ 다"를 예측할 때 모델이 내부적으로 생각한 단어 확률이
다음처럼 정해져 있다고 가정.

top-p=0.9 → 파랗다, 맑다, 흐리다, 높다, 아름답다까지만
후보로 삼아서 그 중 선택 (누적 확률 89%)

단어 후보	확률
파랗다	40%
맑다	30%
흐리다	10%
높다	5%
아름답다	4%
기타	11%

Max-token

답변 길이를 제한하는 파라미터

- 원리

모델이 생성할 수 있는 최대 단위 수(토큰) 제한

- 예시

`max_tokens=5` -> "하늘은 파랗고 맑다." (5 토큰에 해당)

`max_tokens=50` -> "하늘은 파랗고, 구름이 둥둥 떠다니며...(이하 생략)" 길게 생성 가능

***1token 당 한글 1~3자**



OpenAI 활용 실습

=== Test 1: {'temperature': 0.2, 'max_tokens': 30} ===

"하늘은"이라는 표현은 다양한 의미를 가질 수 있습니다. 하늘은 자연의 일부로서, 날씨, 계절

=== Test 2: {'temperature': 0.7, 'max_tokens': 50} ===

"하늘은"이라는 표현은 다양한 의미로 해석될 수 있습니다. 하늘은 자연의 일부로서, 날씨, 계절, 그리고 우리의 감정과 연결될 수 있는 중요한 요소입니다. 하늘을 바라보

=== Test 3: {'temperature': 1.2, 'max_tokens': 70} ===

"하늘은"으로 시작하는 문장은 여러 가지로 이어질 수 있습니다. 예를 들어:

1. 하늘은 맑고 푸르며, 햇살이 눈부시게 빛납니다.
2. 하늘은 별들로 가득 차 있어, 밤하늘이 정말 아름답습니다.

LLM 한계 - 할루시네이션

거짓 정보를 생성하는 현상

원인

확률 기반 생성LLM은 “사실”이 아니라 가장 그럴듯한 단어 조합을 예측함.
→ 실제로 존재하지 않아도 자연스러운 문장을 만들어냄.

해결 방안

- 파인 튜닝, RAG 등

LLM 발전 – 추론 능력

복잡한 문제를 단계적으로 사고하고 답변하는 능력

핵심 원리

확률적 추론 기반으로 학습된 LLM은 언어 패턴 속에서 논리 구조와 인과 관계를 통계적으로 학습함.

→ 명시적 규칙 없이도 “사고 과정(Chain of Thought)”처럼 단계적 reasoning 수행 가능.

LLM 발전 – 멀티모달

텍스트 외의 다양한 형태의 데이터를 이해하고 처리하는 능력

핵심 원리

멀티모달 LLM은 텍스트·이미지·음성·영상 등 서로 다른 입력을 통합적으로 학습함.
→ 서로 다른 정보 간의 연관성을 파악하여 복합적인 의미 이해와 추론이 가능함.

Langchain





RAG

Retrieval Augmented Generation

검색 증강 생성

LLM의 한계점

- 학습하지 않은 데이터에 대해 할루시네이션 현상을 보임.
- 특정 정보에 대해 답변할 수 있으려면 추가 학습(Fine-tuning)이 필요함.
- 추가 학습에는 많은 리소스(데이터, GPU)가 소요됨.

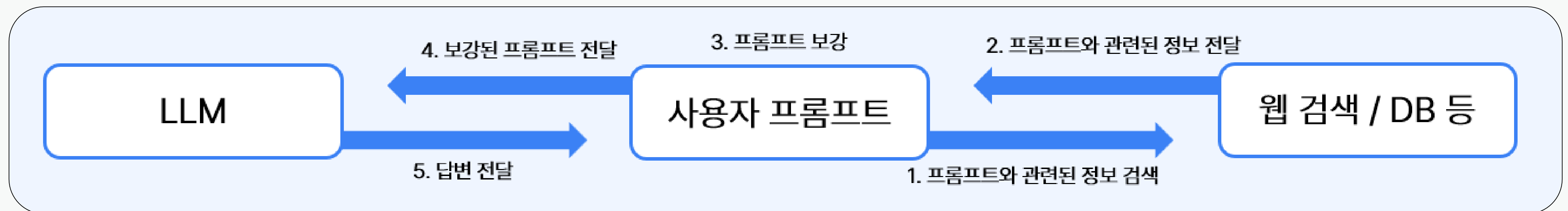
→ Fine-tuning보다 쉽게 특정 정보에 대해 답변하는 방법이 있을까?

RAG란?

검색 증강 생성(Retrieval-Augmented Generation)으로

입력 프롬프트와 유사한 데이터를 검색하여 가져와서

입력 프롬프트를 증강하고 증강된 입력 프롬프트를 통해 답변을 생성하는 기법입니다.



RAG vs. Fine-tuning

- 파인튜닝이란

이미 사전 학습(pretrained)된 모델에 새로운 데이터셋을 주어 **추가 학습을 시키는 방법**입니다.

	파인튜닝	RAG
지식 업데이트	재학습 필요	데이터만 추가하면 즉시 반영
단점	많은 데이터와 GPU 필요	검색기의 퀄리티에 따라 답변 퀄리티가 달라짐
비용	높음 (GPU를 통한 훈련)	낮음 (검색 데이터만 필요)

RAG의 필요성

1. 최신 정보 반영 가능.
2. 할루시네이션 현상 감소
3. 사내 문서 기반 정확한 답변 특정 도메인에 최적화.



최신성

- ✓ LLM 학습 이후 생성된 최신 데이터 활용
- ✓ 실시간 문서/데이터베이스 연동 가능
- ✓ 지속적 업데이트 없이 최신 정보 제공



신뢰성

- ✓ 환각(Hallucination) 현상 최소화
- ✓ 출처 추적 및 정보 검증 가능
- ✓ 구조화된 데이터 기반 정확성 향상

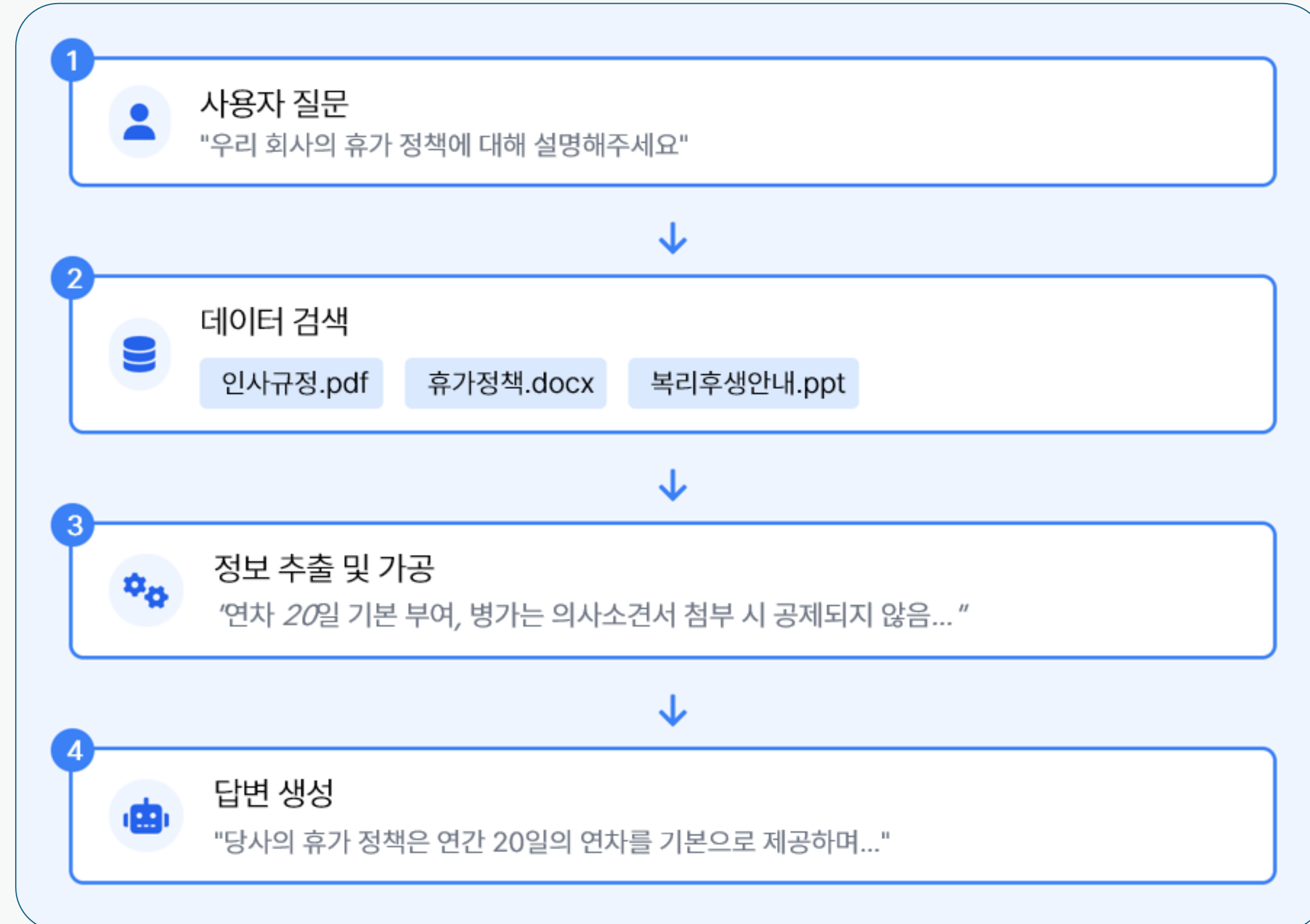


개인화

- ✓ 기업 내부 지식/문서 활용
- ✓ 사용자 컨텍스트에 맞춘 응답 생성
- ✓ 도메인 특화 지식 적용 및 맞춤형 서비스

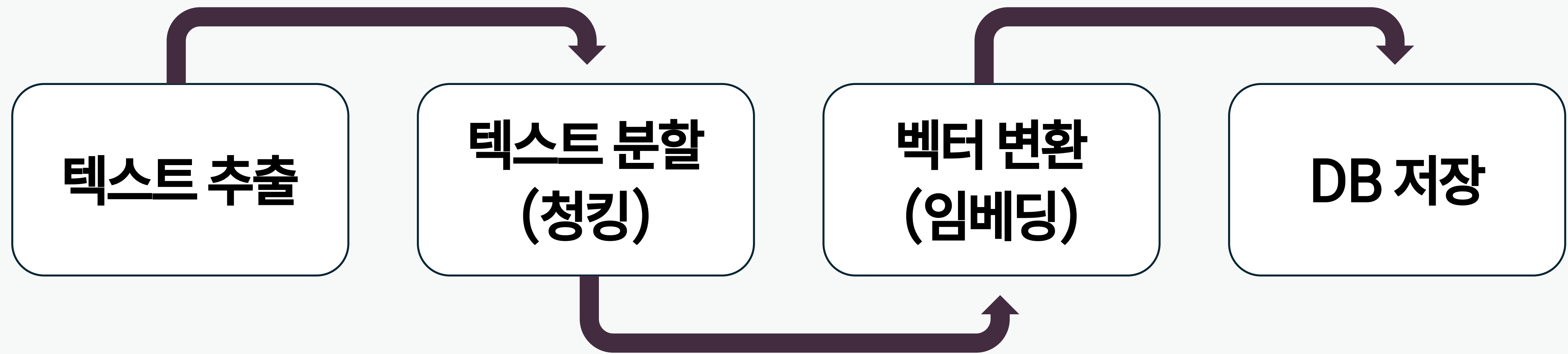
RAG의 기본 구조

RAG는 크게 4단계의 프로세스로 구성됩니다.



벡터 DB

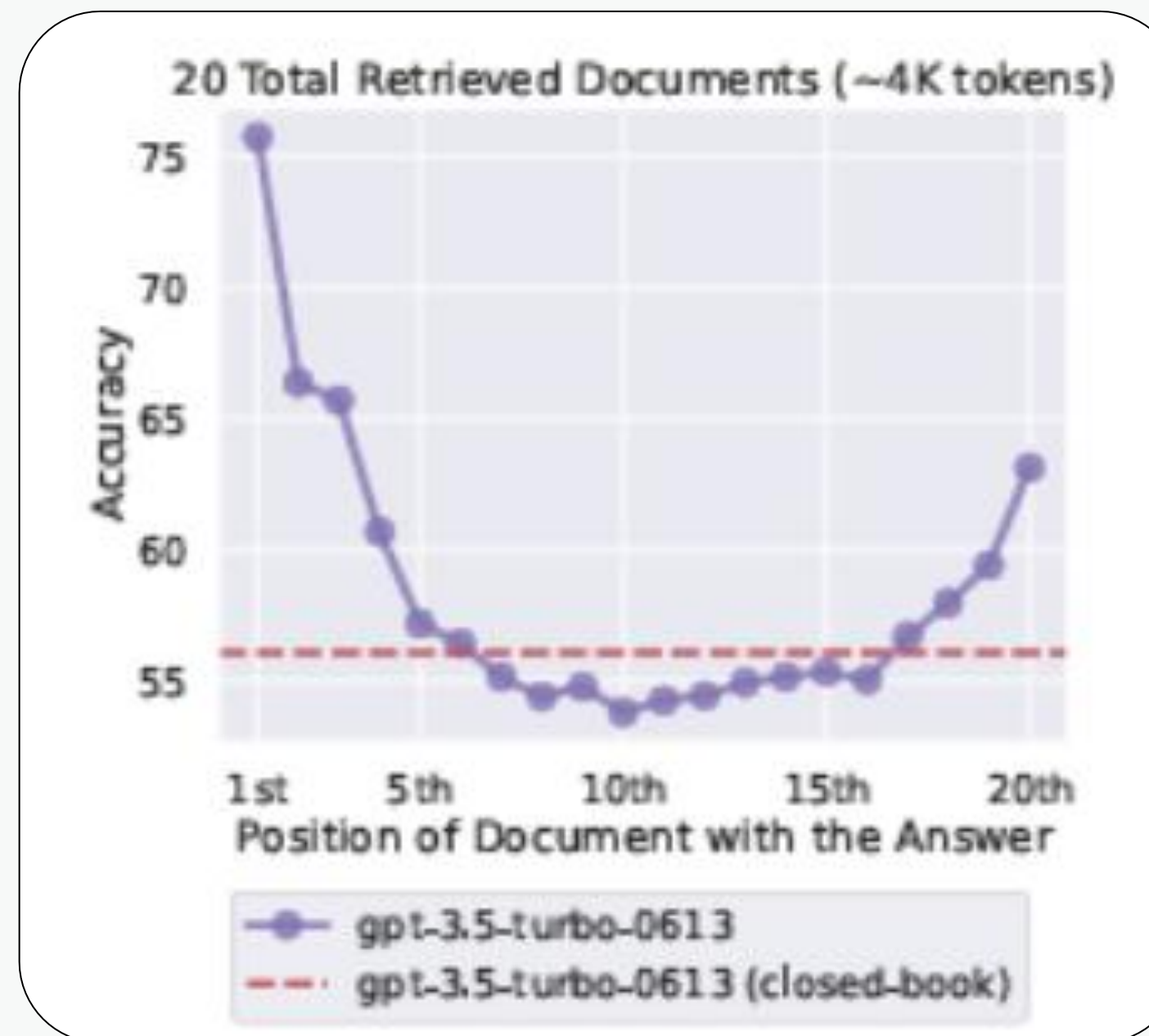
데이터를 벡터로 저장하고, 벡터들 간의 유사도를 빠르게 검색할 수 있도록 설계된 데이터베이스



전체 문서를 다 넣으면 되는거 아니가요?

📦 파일 전체를 넣을 수 없는 이유

1. LLM 입력 프롬프트 길이 제한이 있음.
2. LLM 특성상 입력 프롬프트의 길이가 길어질 때 중간 정보를 소실하는 특성이 있음.



벡터 DB

- 청크 크기와 검색 속도 / 정확도 관계

청크의 크기를 **작게 하면**

입력 프롬프트와 유사한 데이터를 **정확하게 찾을 확률이 높아지지만**

청크의 갯수가 많아지기 때문에 **검색 속도가 느려짐.**

→ 정확도와 속도는 **trade-off 관계**이므로 적절한 값을 찾는 게 중요

검색기

사용자가 입력한 프롬프트와 관련된 정보를 벡터DB에서 찾아주는 역할을 합니다.

1. 코사인 유사도

두 벡터가 이루는 각도를 기준으로 유사도 판단

벡터의 방향만 비교 → 문장 길이의 영향 X

2. 유클리드 거리

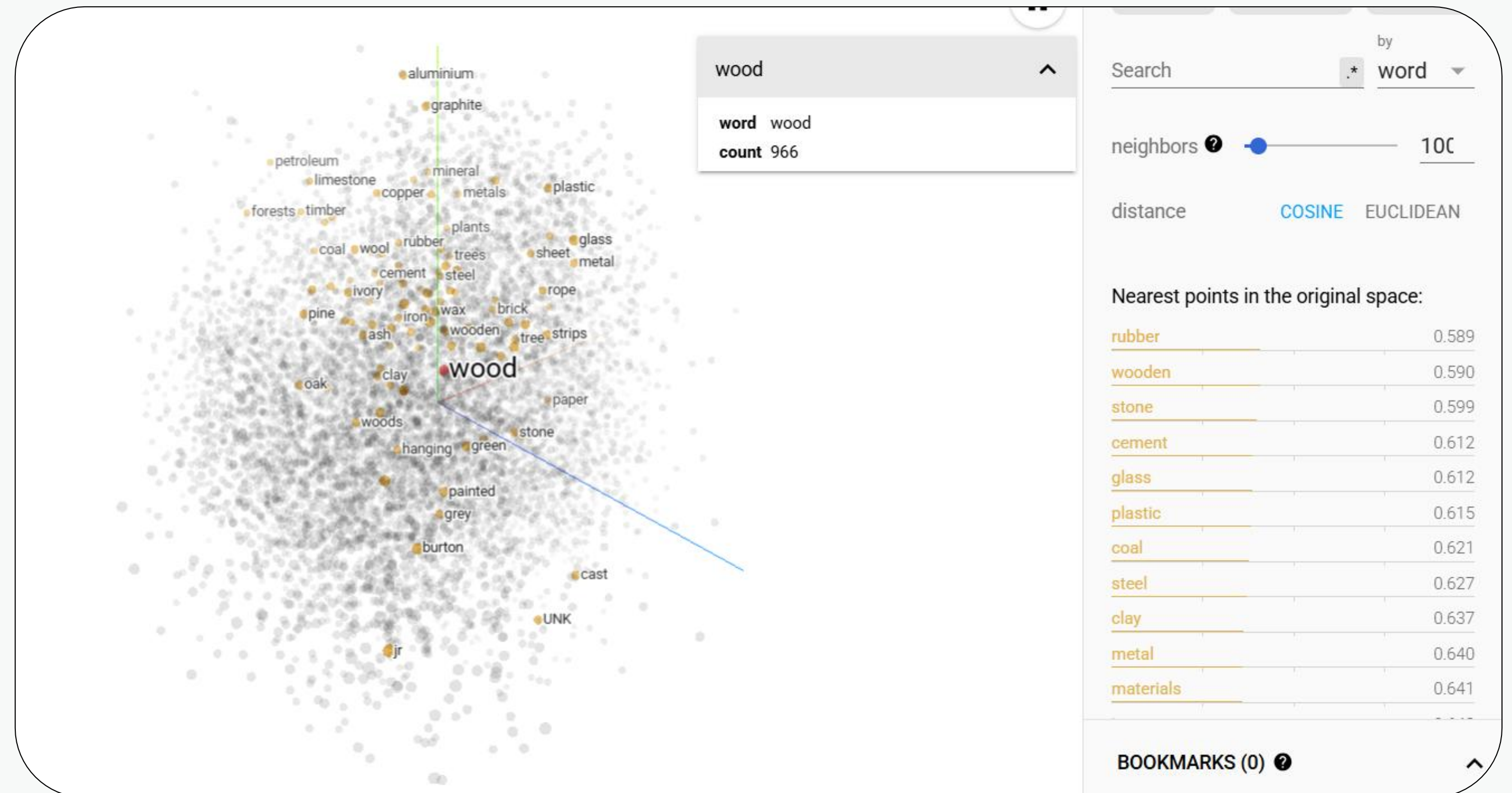
두 벡터 간의 직선 거리

길이, 크기 모두 반영됨

3. 내적

두 벡터의 곱의 총합

둘 다 클수록 유사도도 커짐



In-Context learning

프롬프트 안에 예시를 제공하여 원하는 작업을 유도하는 학습 방식

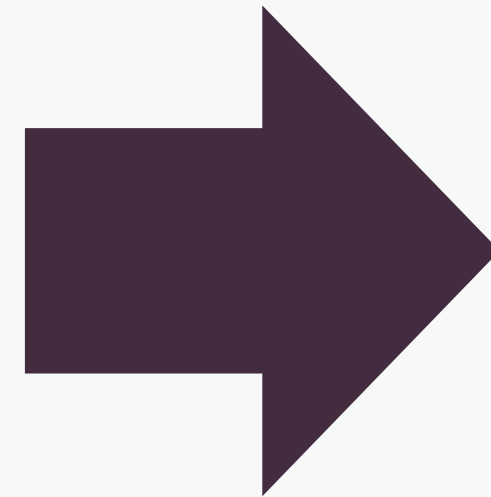
- 입력 프롬프트

[강사]

이름 : 홍길동

MBTI : ENFP

강사의 MBTI가 뭐야?



- 답변

강사의 MBTI는 ENFP입니다.

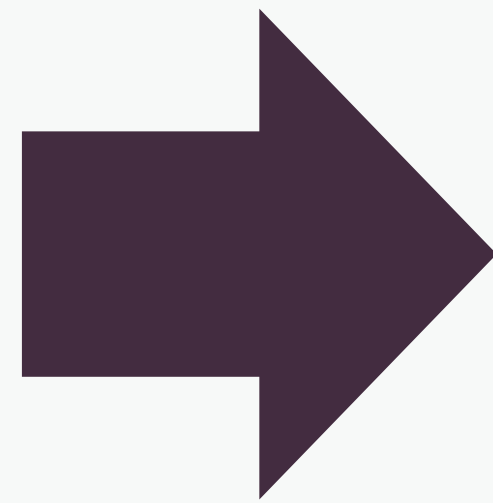
강사의 데이터를 학습하지 않았어도 프롬프트 안의 예시로 관련된 답변이 가능함.

In-Context learning

프롬프트 안에 예시를 제공하여 원하는 작업을 유도하는 학습 방식

- 입력 프롬프트

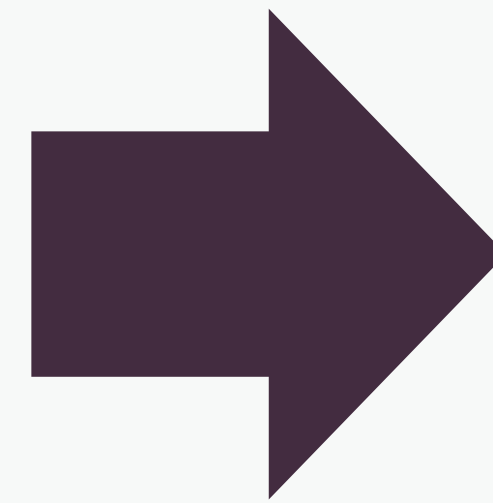
사내 여름 휴가
일수에 대해 설명해줘



- 보강된 프롬프트

[사내 휴가 일수 규정]
1-1 휴가는

사내 여름 휴가
일수에 대해 설명해줘



- 답변

사내 여름 휴가 일수는
다음과 같습니다.

휴가 규정에 의해

❖ 랭체인



LLM을 효율적으로 연결하고 활용하는 파이썬 라이브러리



랭체인 핵심 구성 요소

구성 요소	설명	예시
Prompt	모델에게 전달할 질문이나 지시문	“이 문장을 영어로 번역해줘”
Model(LLM)	GPT 같은 언어 모델	GPT-4, Claude, Gemini 등
Parser	모델의 출력을 원하는 형식으로 가공	JSON, 숫자, 텍스트 등
Chain	위 요소들을 연결한 흐름	Prompt → Model → Parser
Tool	외부 API나 DB와 연결하는 기능	날씨 API, 구글 검색 등



랭체인 패키지 종류

패키지명	설명	주요 구성 요소 / 기능
langchain	LangChain의 기존 통합 패키지	체인, 프롬프트, LLM, 메모리, 툴, 에이전트 등을 모두 포함
langchain_community	커뮤니티 기여 기반 추가 기능 (문서 로더, 벡터 DB 등)	PDF/Docx/CSV 로더, Chroma/Pinecone/MongoDB 벡터 저장소, HuggingFace/ChatGPT API 등
langchain_core	LangChain의 핵심 기능만 분리한 경량 패키지	Prompt, OutputParser, Runnable, BaseLLM 등의 핵심 추상화 제공
langchain_experimental	실험적 기능 모듈, 정식 API 이전의 기능 저장소	로컬 RAG, MultiModalChain, GraphDocument 등의 연구중 기능

LLM

사용자 입력(프롬프트)을 받아 응답을 생성하는 객체

```
from langchain_openai import ChatOpenAI

llm = ChatOpenAI(model="gpt-3.5-turbo", api_key="YOUR_OPENAI_API_KEY")
llm.invoke("안녕! 오늘 날씨 어때?")
```

invoke, batch, stream을 이용하여 입력 프롬프트를 전달합니다.

Prompt Template

프롬프트를 구조적으로 정의하고 사용자 입력을 바인딩하여 LLM에 전달

```
from langchain.prompts import PromptTemplate

template = "다음 문장을 영어로 번역해줘: {sentence}"
prompt = PromptTemplate.from_template(template)
final_prompt = prompt.format(sentence="안녕하세요, 만나서 반가워요.")
```

프롬프트 템플릿을 구성하여 벡터 DB에서 정보를 가져와 완성된 프롬프트를 LLM에게 전달하는 역할을 합니다.

Splitter

문서나 텍스트를 일정 크기로 나누어 벡터화하기 쉽게 만듦

```
from langchain.text_splitter import CharacterTextSplitter

text = "이것은 긴 문장입니다. 문장을 분할해보겠습니다."
splitter = CharacterTextSplitter(separator=".", chunk_size=10, chunk_overlap=0)
chunks = splitter.split_text(text)
```

- 문맥, 특정 기호, 글자 수 기준으로 텍스트를 나누는 역할을 합니다.
- 문서 유형에 맞춰 테스트를 해보며 적절한 청크 크기를 정합니다.

Embedding

나누어진 문서들을 벡터로 변환

```
from langchain_openai import OpenAIEmbeddings  
  
embeddings = OpenAIEmbeddings(api_key="YOUR_OPENAI_API_KEY")
```

Vector DB

문서를 임베딩 벡터로 변환하여 저장하고, 검색 시 유사 문서를 반환

```
from langchain_community.vectorstores import Chroma
from langchain_openai import OpenAIEmbeddings

embeddings = OpenAIEmbeddings(api_key="YOUR_OPENAI_API_KEY")

db = Chroma.from_texts(
    chunks,
    embedding=embeddings,
)
```

임베딩 모델을 이용하여 텍스트 조각들을 벡터로 변환하여 저장합니다.

Retriever

유저의 입력 프롬프트에 대해 VectorDB에서 유사한 문서를 반환

```
retriever = db.as_retriever(search_kwargs={"k": 3})  
results = retriever.invoke("AI에 대해 알려줘")
```

유저의 입력 프롬프트와 유사한 문서를 설정한 K개만큼 검색할 수 있습니다.

LangChain Expression Language (LCEL)

랭체인 요소들을 파이프기호로 연결하여 입력을 받고 출력을
다음 요소로 전달하는 역할

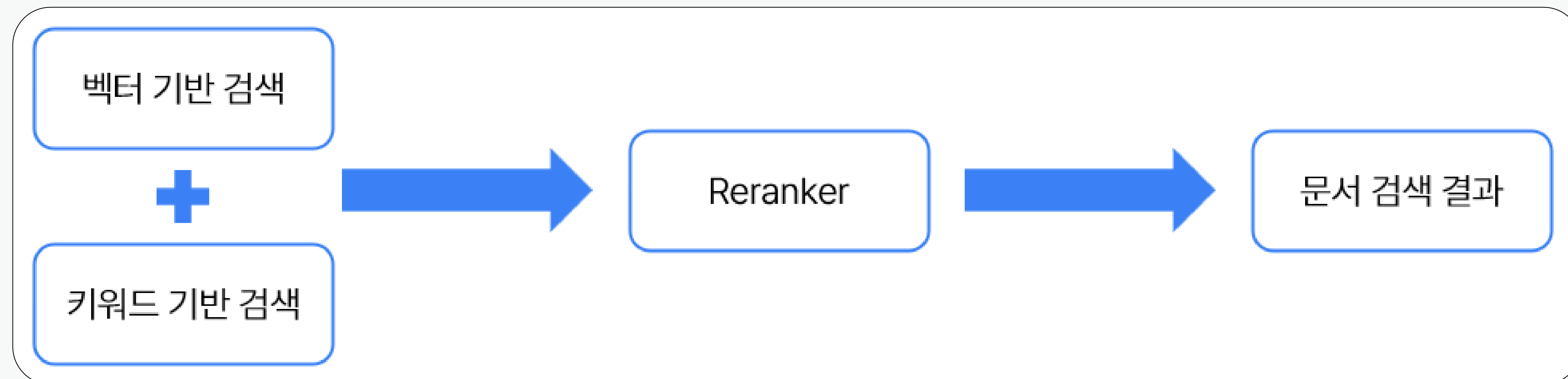
```
rag_chain = (  
    {"context": compression_retriever, "question": RunnablePassthrough()}  
    | prompt  
    | llm  
    | StrOutputParser()  
)
```

❖ 벡터 기반 Retriever의 한계

벡터 기반의 Retriever는 속도가 빠르고 확장성이 크지만,
단순 유사도 기반이라 미세한 차이를 잘 잡지 못함.

해결 방법

1. 키워드 검색 방법과 함께 하이브리드 방식으로 사용.
2. Retrieval 후 Reranker 진행



Reranker

쿼리와 문서를 동시에 입력받아 문맥적 의미와 정확도를 정밀하게 파악합니다.

- 원리

‘질문-문서 쌍’을 하나씩 교차 분석해 관련성 점수를 계산 → 점수에 따라 문서의 순위를 다시 매김.
트랜스포머 기반 교차 인코더(Cross Encoder) 구조 모델을 사용.

- 장점

검색 정확도 개선, 복잡한 의미 관계의 파악, RAG 시스템 전반의 답변 품질 향상.

- 단점

계산 비용과 시간이 늘어나기 때문에, 보통 1차 필터링을 거친 상위 5~10개 문서에 제한해서 적용.

LLM & Embedding



ChatOpenAI()

라이브러리 설치

```
!pip install langchain_openai
```

LLM & Embeddings 객체 생성

```
from langchain_openai import ChatOpenAI, OpenAIEmbeddings

# ChatOpenAI 객체 생성: OpenAI의 gpt-4o-mini 모델을 사용하도록 설정
llm = ChatOpenAI(model='gpt-4o-mini', openai_api_key="OPENAI_API_KEY")

# OpenAIEmbeddings 객체 생성: 텍스트를 벡터로 변환(Embedding)하는 데 사용
embeddings = OpenAIEmbeddings(openai_api_key="OPENAI_API_KEY")
```


Ollama



Cloud models are now available in Ollama

**Chat & build with
open models**

로컬 환경에서 대형 언어 모델을 쉽게 설치하고 실행할 수 있는 오픈소스 플랫폼



Ollama 다운로드



[Cloud models](#) are now available in Ollama

Chat & build with
open models

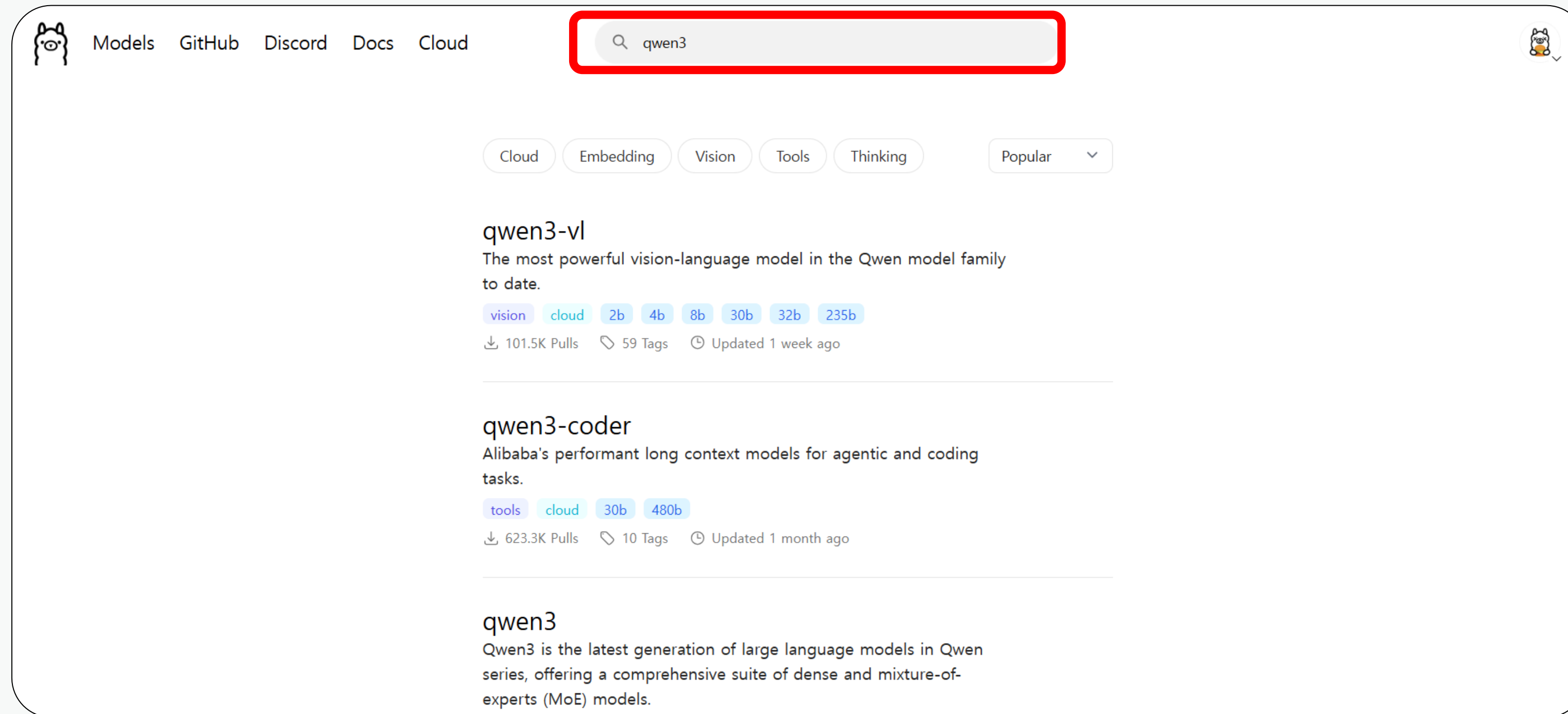
Download

Available for macOS,
Windows, and Linux

Ollama 페이지에서 다운로드



오픈소스 모델 검색



검색 창에서 오픈소스 모델 검색



모델 확인

qwen3

ollama run qwen3

12.7M Downloads

Updated 3 weeks ago

Qwen3 is the latest generation of large language models in Qwen series, offering a comprehensive suite of dense and mixture-of-experts (MoE) models.

tools

thinking

0.6b

1.7b

4b

8b

14b

30b

32b

235b

Models

View all →

Name	Size	Context	Input
qwen3:latest	5.2GB	40K	Text
qwen3:0.6b	523MB	40K	Text
qwen3:1.7b	1.4GB	40K	Text
qwen3:4b	2.5GB	256K	Text
qwen3:8b latest	5.2GB	40K	Text
qwen3:14b	9.3GB	40K	Text
qwen3:30b	19GB	256K	Text

크기별 모델 확인 가능

Langchain - Ollama - LLM

모델 다운로드

```
# qwen3의 1.7b 크기 모델 다운로드  
!ollama pull qwen3:1.7b
```

LLM 객체 생성

```
from langchain_community.chat_models import ChatOllama  
  
# Ollama 모델을 불러옵니다.  
llm = ChatOllama(model="qwen3:1.7b")
```


Langchain – Ollama - Embeddings

모델 다운로드

```
# bge-m3의 567m 크기 모델 다운로드  
!ollama pull bge-m3:567m
```

Embeddings 객체 생성

```
from langchain_community.embeddings import OllamaEmbeddings  
  
# Ollama 모델을 불러옵니다.  
embeddings = OllamaEmbeddings(model="bge-m3:567m")
```

PromptTemplate



PromptTemplate

단일 문자열 기반 프롬프트에 변수(플레이스홀더)를 삽입해 동적으로 문장을 생성할 수 있도록 하는 템플릿

```
from langchain_core.prompts import PromptTemplate

# 템플릿 문자열 정의
# {input}은 나중에 실제 입력값으로 대체될 입력 변수
template = "{input}에 대해 설명해주세요."

# 템플릿 문자열로부터 PromptTemplate 객체 생성
# from_template() 메서드는 템플릿 문자열을 기반으로 자동으로 입력 변수 추론
prompt = PromptTemplate.from_template(template)

# 생성된 PromptTemplate 객체 출력 (정의된 템플릿 구조 확인 가능)
prompt
```


ChatPromptTemplate

여러 역할(system, human, ai)로 구성된 대화형 메시지들을 템플릿 형태로 정의하여 대화형 모델에 전달할 프롬프트를 구성

```
from langchain_core.prompts import ChatPromptTemplate

# ChatPromptTemplate 객체를 메시지 목록으로부터 생성
# from_messages() 메서드를 통해 여러 역할(role)과 내용(content)으로 구성된 대화 메시지 목록을 정의할 수 있습니다.
template = ChatPromptTemplate.from_messages(
    [
        # system 역할: AI의 기본 성격, 말투, 행동 방식을 설정
        ("system", "당신은 화가 난 상사입니다. 면박을 주며 설명을 합니다."),

        # human 역할: 사용자가 입력하는 메시지를 나타냄
        ("human", "{input}"),
    ]
)
```


ChatPromptTemplate

Message Role

역할(Role)	설명	사용 목적
system	AI의 전반적인 태도, 성격, 규칙을 정의하는 메시지	행동 방식을 지시하고 일관성 있게 유지하도록 설정
human	실제 사용자 또는 질문을 던지는 인간의 메시지	LLM에 질문·명령·요청 등을 전달
ai	AI가 사용자에게 응답하는 메시지	모델이 생성한 답변을 나타냄

Document Loader



PDF Loader

- 라이브러리 설치

```
!pip install pypdf
```

- 코드

```
from langchain_community.document_loaders import PyPDFLoader

# 로드할 PDF 파일 경로를 지정
# 예: FILE_PATH = "example.pdf"
loader = PyPDFLoader(FILE_PATH)

# PDF 파일을 로드하여 문서(Document) 객체 리스트로 반환
# 각 페이지가 하나의 문서로 처리되어 리스트 형태로 저장됨
docs = loader.load()
```

Word Loader

-라이브러리 설치

```
!pip install docx2txt
```

- 코드

```
from langchain_community.document_loaders import Docx2txtLoader

# 로드할 Word 파일 경로를 지정하여 Docx2txtLoader 객체 생성
# 예: FILE_PATH = "example.docx"
loader = Docx2txtLoader(FILE_PATH)

# 지정한 Word 문서를 로드하여 Document 객체 리스트로 반환
# 문서의 텍스트 내용이 하나의 Document 객체로 저장됨
docs = loader.load()
```


Csv Loader

- 특징

각 행이 하나의 문서로 처리됨.

- 코드

```
from langchain_community.document_loaders.csv_loader import CSVLoader

# CSV 파일 경로를 지정하여 CSVLoader 객체 생성
# 예: FILE_PATH = "example.csv"
loader = CSVLoader(FILE_PATH)

# CSV 파일을 로드하여 Document 객체 리스트로 반환
# 기본적으로 CSV의 각 행이 하나의 문서로 처리됨
docs = loader.load()
```

Textsplitter



CharacterTextSplitter

- 라이브러리 설치

```
!pip install langchain-text-splitters
```

- 코드

```
from langchain_text_splitters import CharacterTextSplitter

# CharacterTextSplitter 클래스는 텍스트 데이터를 특정 기준에 따라 잘게 나누는 도구입니다.
# 긴 문서를 LLM이 처리하기 좋도록 일정한 크기로 분할(chunking)하는 데 사용됩니다.

text_splitter = CharacterTextSplitter(
    separator=" ",          # 텍스트를 분리할 기준 문자. 여기서는 공백(" ") 단위로 쪼갬.
    chunk_size=250,         # 한 청크의 최대 문자 수. 단위는 '문자 수', 250자씩 잘라냄.
    chunk_overlap=50        # 청크 간 중복되는 문자 수. 앞 청크의 끝 50자를 다음 청크의 시작에 포함.
)
```


RecursiveCharacterTextSplitter

- 특징

문단 → 줄바꿈 → 공백 → 문자를 기준으로 단계적 분리하는 TextSplitter

- 코드

```
from langchain_text_splitters import RecursiveCharacterTextSplitter

# RecursiveCharacterTextSplitter:
# 텍스트를 '의미 단위'에 가깝게, 큰 단위 → 작은 단위 순으로 재귀적으로 분할하는 스플리터
text_splitter = RecursiveCharacterTextSplitter(
    chunk_size=200,    # 한 청크의 최대 길이(문자 수 기준) - 200자씩 나눔
    chunk_overlap=20   # 청크들 간 겹치는 길이 - 20자씩 앞 청크 끝부분을 포함
)
```

VectorDB



Chroma DB 구축

- 라이브러리 설치

```
!pip install -q langchain_chroma
```

- 코드

```
from langchain_chroma import Chroma

# 문서 데이터(docs)를 기반으로 Chroma 벡터스토어 생성
chroma = Chroma.from_documents(
    docs,                                # 임베딩할 문서 리스트 (예: 텍스트 또는 Document 객체)
    embedding=embeddings                 # 사용할 임베딩 모델 (예: OpenAIEmbeddings, HuggingFaceEmbeddings 등)
)
```

벡터 DB를 저장하는 이유

문서 많으면 벡터DB 처음부터 만드는 데 시간 오래 소요.

한 번 생성한 벡터DB 로컬에 저장해두고 필요할 때마다 불러와 사용.
매번 임베딩 안 해도 되어 시간, 비용 아낄 수 있음.

Faiss DB 저장 및 로드

- 코드

```
# 생성된 벡터스토어를 로컬 경로에 저장
faissdb.save_local("./faiss") # 폴더가 없으면 자동 생성됨

# 저장된 벡터스토어를 다시 로드
new_faissdb = FAISS.load_local(
    "./faiss", # 저장된 벡터스토어 경로
    embeddings=embeddings # 로드 시 동일한 임베딩 모델을 지정해야 함
)
```

Chroma DB 저장 및 로드

- 코드

```
# 문서 데이터(docs)를 기반으로 Chroma 벡터스토어 생성
chroma = Chroma.from_documents(
    docs,          # 임베딩할 문서 리스트 (예: 텍스트 또는 Document 객체)
    embedding=embeddings, # 사용할 임베딩 모델 (예: OpenAIEmbeddings, HuggingFaceEmbeddings 등)
    persist_directory="./chroma" # 벡터스토어를 저장할 로컬 디렉토리
)
```

```
# 기존에 저장된 Chroma 벡터스토어를 로드
new_chroma = Chroma(
    embedding_function=embeddings, # 벡터 임베딩에 사용할 임베딩 모델 (저장 시 사용했던 것과 동일해야 함)
    persist_directory="./chroma" # Chroma 벡터스토어가 저장된 로컬 디렉토리
)
```

Retriever



as_retriever() 주요 파라미터

- search_kwargs

k: 검색할 문서 개수 (기본값: 4)

- 코드

```
# vectorstore를 검색기(retriever)로 변환하면서 검색 옵션 설정
# - search_kwargs={"k": 5} : 유사도가 가장 높은 상위 5개의 문서를 반환하도록 설정
# - faiss_retriever2은 이후 사용자의 질문(쿼리)에 대해 관련 문서를 최대 5개까지 찾아주는 역할 수행
faiss_retriever2 = vectorstore.as_retriever(search_kwargs={"k": 5})
```

BM25Retriever

BM25 알고리즘으로 키워드 기반 문서 검색을 수행하는 검색기

- 라이브러리 설치

```
!pip install rank_bm25
```

- 코드

```
from langchain.retrievers import BM25Retriever

# BM25 알고리즘을 기반으로 문서 검색을 수행하는 검색기 생성
# - BM25Retriever.from_documents(docs): 텍스트 기반 키워드 검색을 위해 문서 리스트(docs)를 로드하여 BM25 검색기 생성
bm25_retriever = BM25Retriever.from_documents(docs)

# 검색 결과로 반환할 문서 수를 3개로 설정
# - k는 반환할 문서의 수를 의미하며, 유사도 순으로 상위 3개 문서를 반환함
bm25_retriever.k = 3
```

EnsembleRetriever

- 특징

여러 검색기의 결과를 가중치 기반으로 조합해 더 정확한 문서 검색을 수행하는 통합 검색기

- 코드

```
# 서로 다른 검색기들을 결합하여 하나의 통합 검색기로 만드는 EnsembleRetriever 생성
# - retrievers=[retriever, bm25_retriever] : 벡터 기반 검색기(retriever)와 BM25 키워드 검색기(bm25_retriever)를 함께 사용
# - weights=[0.4, 0.6] : 두 검색기의 결과에 가중치를 부여해 점수를 계산 (벡터 검색 40%, BM25 검색 60%)
# → 두 방식의 장점을 결합해 더 정확한 검색 결과를 얻기 위한 방식
ensemble_retriever = EnsembleRetriever(
    retrievers=[retriever, bm25_retriever],
    weights=[0.4, 0.6]
)
```


Reranker



Reranker

```
from langchain.retrievers import ContextualCompressionRetriever
from langchain.retrievers.document_compressors import CrossEncoderReranker
from langchain_community.cross_encoders import HuggingFaceCrossEncoder

# ✅ 1. 사전 학습된 Cross-Encoder 모델 로드
# - "BAAI/bge-reranker-v2-m3" 모델은 문서의 중요도를 점수로 매겨 재정렬(reranking)하는 데 사용됨
model = HuggingFaceCrossEncoder(model_name="BAAI/bge-reranker-v2-m3")

# ✅ 2. 검색된 문서들 중에서 더 관련성이 높은 문서를 골라 재정렬해주는 reranker 생성
# - top_n=3 : 최종적으로 상위 3개의 문서만 남기도록 압축(compression) 수행
compressor = CrossEncoderReranker(model=model, top_n=3)

# ✅ 3. 기존 retriever 결과를 받아 reranker로 문서를 압축(필터링/재정렬)하는 검색기 생성
# - base_retriever=retriever : 먼저 일반 retriever가 문서를 검색
# - base_compressor=compressor : 검색된 문서를 재평가하여 가장 중요한 문서만 남김
compression_retriever = ContextualCompressionRetriever(
    base_compressor=compressor,
    base_retriever=retriever
```


Output-parser



Output-parser

- StrOutputParser()

LLM의 전체 응답을 단순 문자열(String) 형태로 반환하는 출력 파서

- JsonOutputParser()

LLM 응답을 JSON 형태로 정규화해주는 출력 파서

JSON

데이터를 저장하고 주고받기 위한 경량 텍스트 기반의 데이터 포맷

- 특징

키-값(key-value) 구조로 데이터를 저장

구조는 중괄호 {} 와 대괄호 [] 로 객체와 배열을 표현

```
{  
  "name": "민수",  
  "age": 30,  
  "skills": ["Python", "JavaScript", "AI"],  
  "active": true  
}
```


Memory



ConversationBufferMemory

대화 내용을 모두 저장하고, 매번 전체 히스토리를 LLM에 전달하는 기본 메모리 모듈.

- 코드

```
# 대화 내용을 메모리에 저장하기 위한 ConversationBufferMemory 객체 생성
memory = ConversationBufferMemory(
    return_messages=True,          # 메시지를 객체 형태로 반환 (True: 전체 메시지 객체 반환)
    memory_key="chat_history"      # 프롬프트에서 사용할 메모리 키 이름
)
```

ConversationBufferWindowMemory

최근 N개의 대화만 저장해 메모리 사용량을 줄이는 윈도우 방식 메모리.

- 코드

```
# ConversationBufferWindowMemory:  
# 최근 k개의 메시지만을 유지하는 대화 메모리입니다.  
# k=3 이므로, 마지막 3개의 메시지만 기억하며, 이전 메시지는 자동으로 삭제됩니다.  
# return_messages=True는 반환 시 메시지를 텍스트가 아닌 메시지 객체 형태로 제공합니다.  
memory = ConversationBufferWindowMemory(k=3, return_messages=True, memory_key="chat_history")
```


ConversationSummaryMemory

진행된 대화를 요약해 저장하고, 오래된 내용을 자동으로 간단하게 정리해주는 메모리.

- 코드

```
# ConversationSummaryMemory:
# 대화가 길어질 경우 모든 메시지를 저장하면 메모리나 토큰 수가 지나치게 커질 수 있습니다.
# ConversationSummaryMemory는 이전 대화를 요약해 저장함으로써 메모리 사용을 줄이고,
# LLM이 긴 대화 내역도 효율적으로 기억할 수 있도록 도와줍니다.
memory = ConversationSummaryMemory(
    llm=llm,                # 대화 요약에 사용할 언어 모델(Large Language Model) 설정
    return_messages=True,   # True로 설정 시, 요약된 메시지들을 messages 형식으로 반환
    memory_key="chat_history" # 메모리에서 저장된 내용을 불러올 때 사용할 키 이름
)
```


Tool



Tool

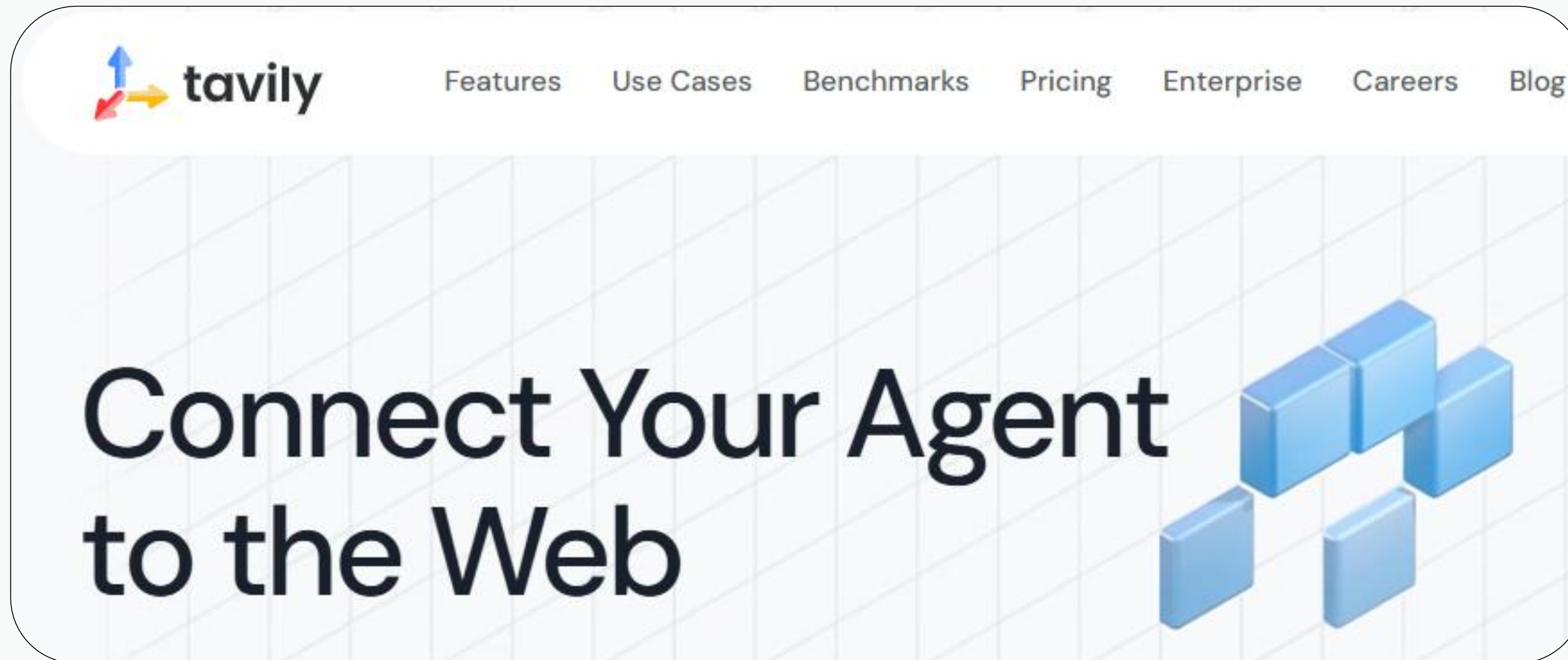
데이터를 저장하고 주고받기 위한 경량 텍스트 기반의 데이터 포맷

- 특징

키-값(key-value) 구조로 데이터를 저장

구조는 중괄호 {} 와 대괄호 [] 로 객체와 배열을 표현

Tavily



실시간 웹 정보를 검색·추출·요약해 주는 AI 검색 엔진

@tool

```
@tool
def plot_csv_graph(params: str) -> str:
    """
    전역 CSV_PATH 경로의 CSV 파일에서 지정된 2개의 컬럼으로 산점도를 생성합니다.
    예: "Age:Fare" → x축: Age, y축: Fare
    """
    try:
        global CSV_PATH
        df = pd.read_csv(CSV_PATH)

        # ✅ 컬럼명 파싱
        parts = params.split(":")
        if len(parts) != 2:
            return "❌ 입력 형식이 잘못되었습니다. 예: 'Age:Fare'"
    except Exception as e:
        return f"오류 발생: {e}"
```

함수를 LangChain에서 사용 가능한 Tool로 등록하기 위해 붙이는 데코레이터

.bind_tools()

```
# 📌 예시 1: 그래프 생성 요청  
# "data.csv" 파일의 "매출" 컬럼을 그래프로 시각화  
tools = [plot_csv_graph, analyze_csv_data]  
llm_with_tools = llm.bind_tools(tools)  
llm_with_tools.invoke({"input": "titanic.csv 파일의 age 컬럼을 그래프로 보여줘"})
```

```
[{'id': 'call_lqgxgn9PHaHXTHaU5hfVlexu',  
  'function': {'arguments': '{"column": "Age"}', 'name': 'plot_csv_graph'},  
  'type': 'function'}]
```

tool을 사용하기 위한 output을 생성하는 메서드

ToolNode()

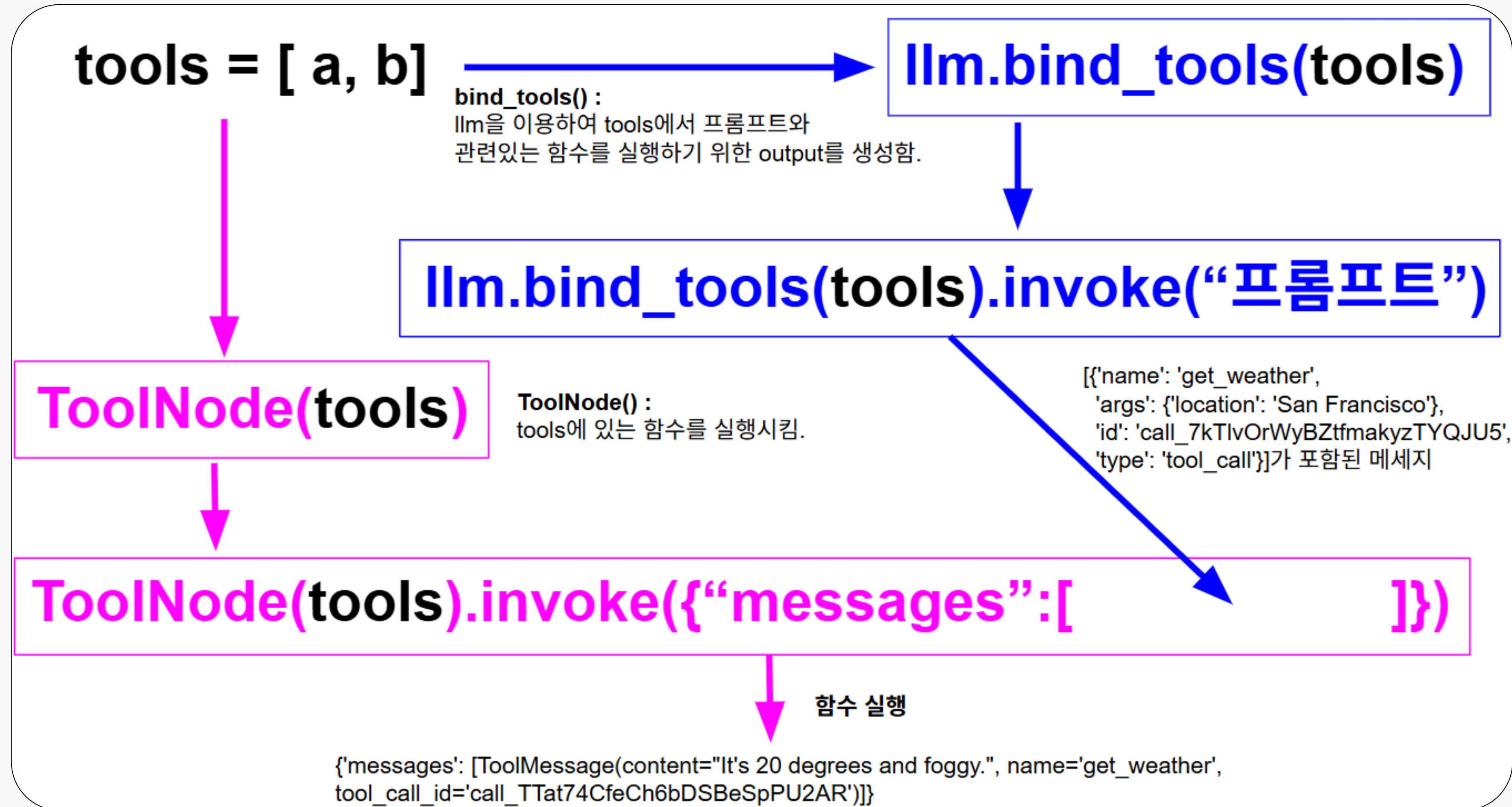
```
from langgraph.prebuilt import ToolNode

# ✅ 5. ToolNode 생성 (명령서 실행자)
tool_node = ToolNode(tools)
# ✅ 6. 위에서 생성된 tool_call_message를 실행
result = tool_node.invoke(tool_call_message.tool_calls)
result
```

```
('{'messages': [ToolMessage(content="📊 'Pclass' 컬럼 요약- 평균: 2.31- 최소값: 1.0- '
최대값: 3.0- 표준편차: 0.84- 결측치 개수: 0- 총 행 수: 891- 고유값 개수: 3",
name='analyze_csv_data', tool_call_id='call_q7Rd8hTRJbut3BQi47L6XduG')])
```

tool 호출을 실행하는 노드

bind_tools() & ToolNode() 순서



LLM이 tool을 선정하는 기준

1. tool 함수의 이름

2. tool 함수의 설명 docstring

```
@tool  
def plot_csv_graph(params: str) -> str:
```

```
"""
```

전역 CSV_PATH 경로의 CSV 파일에서 지정된 2개의 컬럼으로 산점도를 생성합니다.
예: "Age:Fare" → x축: Age, y축: Fare

```
"""
```

```
try:
```

```
    global CSV_PATH
```

```
    df = pd.read_csv(CSV_PATH)
```


Agent



AI Agent

외부 환경·Tool과 상호작용하며 복잡한 태스크를 해결하는 주체적 LLM 시스템

추론 능력

**Tool
실행 능력**

기억 능력

사용자 목적을 **이해**하고,

스스로 필요한 작업을 **계획**하고,

API 호출, 파일 생성, 검색, 데이터 처리 등의 **도구 사용**을 통해

최종 결과물을 제공할 수 있는 **자율적 AI 시스템**

ReAct 구조

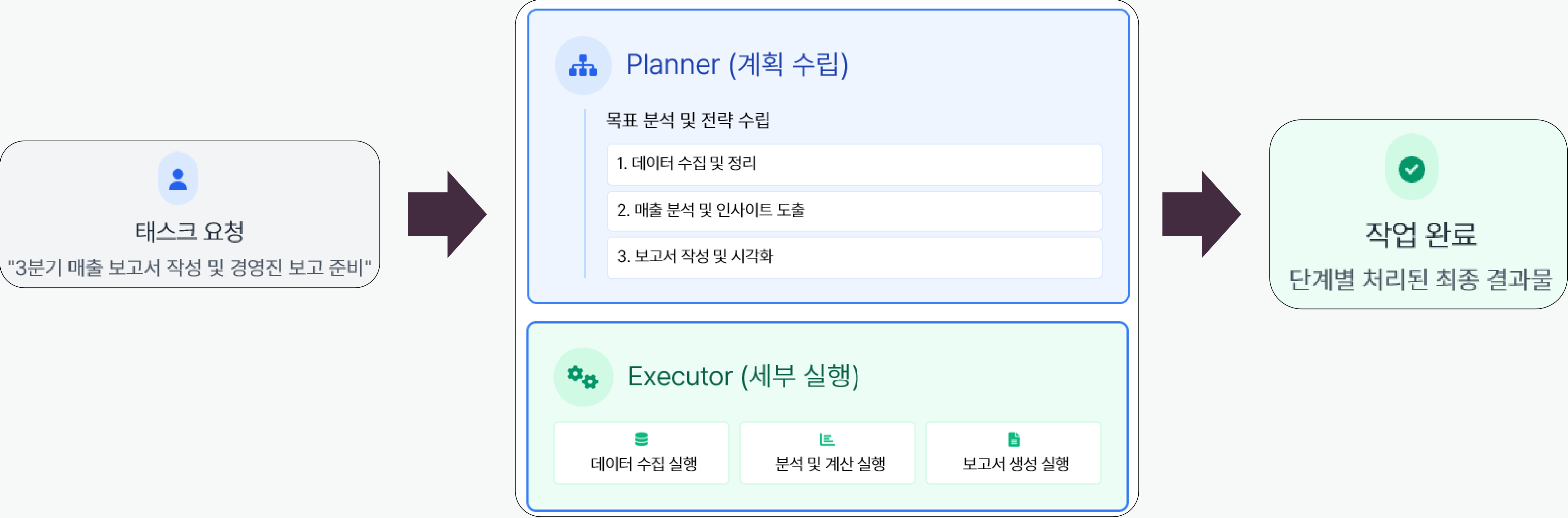
LLM이 추론과 행동을 번갈아 수행하며 복잡한 문제를 해결 하는 구조

ReAct의 핵심 과정

- 1 **추론(Reasoning)**: 문제 상황 분석 및 다음 행동 계획
- 2 **행동(Acting)**: 추론에 따른 도구 활용 및 행동 수행
- 3 **관찰(Observing)**: 행동 결과를 관찰하고 피드백 획득

❏ 계획 수립 / 세부 실행 구조

계획 수립과 세부 실행의 분리 구조를 통해 복잡한 작업을 효율적으로 처리



 AI Agent vs 챗봇

구분	기존 챗봇	AI Agent
작동 방식	질문 → 답변	목표 → 계획 → 실행
역할	대화만 수행	대화 + 작업 수행
지능 수준	수동적 반응형	능동적 자율형
도구 사용	불가 또는 제한적	외부 도구 활용 가능

create_react_agent

ReAct 패턴을 기반으로 LLM이 스스로 사고하고 적절한 툴을 선택해 문제를 해결하도록 설계된 범용 에이전트

- 코드

```
agent = create_react_agent(  
    llm=llm, # 사용할 LLM (예: ChatOpenAI 등), 반드시 사전에 정의되어 있어야 함  
    tools=tools, # 에이전트에 등록할 도구 목록  
    prompt=prompt, # ReAct 프롬프트 구조를 따르는 템플릿  
)
```

```
executor = AgentExecutor(  
    agent=agent, # 위에서 생성한 ReAct 에이전트  
    tools=tools, # 에이전트가 사용할 도구들을 재차 등록  
    verbose=True # 실행 시 에이전트의 내부 추론 과정을 상세히 출력 (디버깅 목적에 유용)  
)
```

create_react_agent - prompt

ReAct 에이전트용 프롬프트에는 아래 변수가 반드시 포함되어야 함.

{input}: 사용자 입력

{agent_scratchpad}: 이전 Thought/Action/Observation 히스토리

{tools}: 사용 가능한 도구 리스트

{tool_names}: 툴 이름 목록



create_react_agent – prompt 예시

```
prompt = PromptTemplate.from_template("""
당신은 주어진 도구를 사용하여 사용자의 질문에 답하는 에이전트입니다.

사용 가능한 도구:
{tools} # 사용 가능한 도구 목록이 문자열로 대체됩니다.

질문: {input} # 사용자가 입력한 질문

다음 형식에 따라 답하십시오:

Thought: 무엇을 할지 생각하세요
Action: 사용할 도구 이름을 작성하세요 [{tool_names}]
Action Input: 입력값
Observation: 실행 결과
... (이 Thought/Action/Action Input/Observation은 반복될 수 있습니다)
Thought: 최종 답을 결정하세요
Final Answer: 사용자에게 줄 최종 답변

{agent_scratchpad} # 이전 단계의 Thought/Action 등의 기록이 여기에 나타납니다.
""")
```

create_csv_agent

CSV 파일의 데이터를 불러와 LLM 기반으로 질의, 분석, 계산을 수행할 수 있는 에이전트

- 코드

```
from langchain_experimental.agents import create_csv_agent

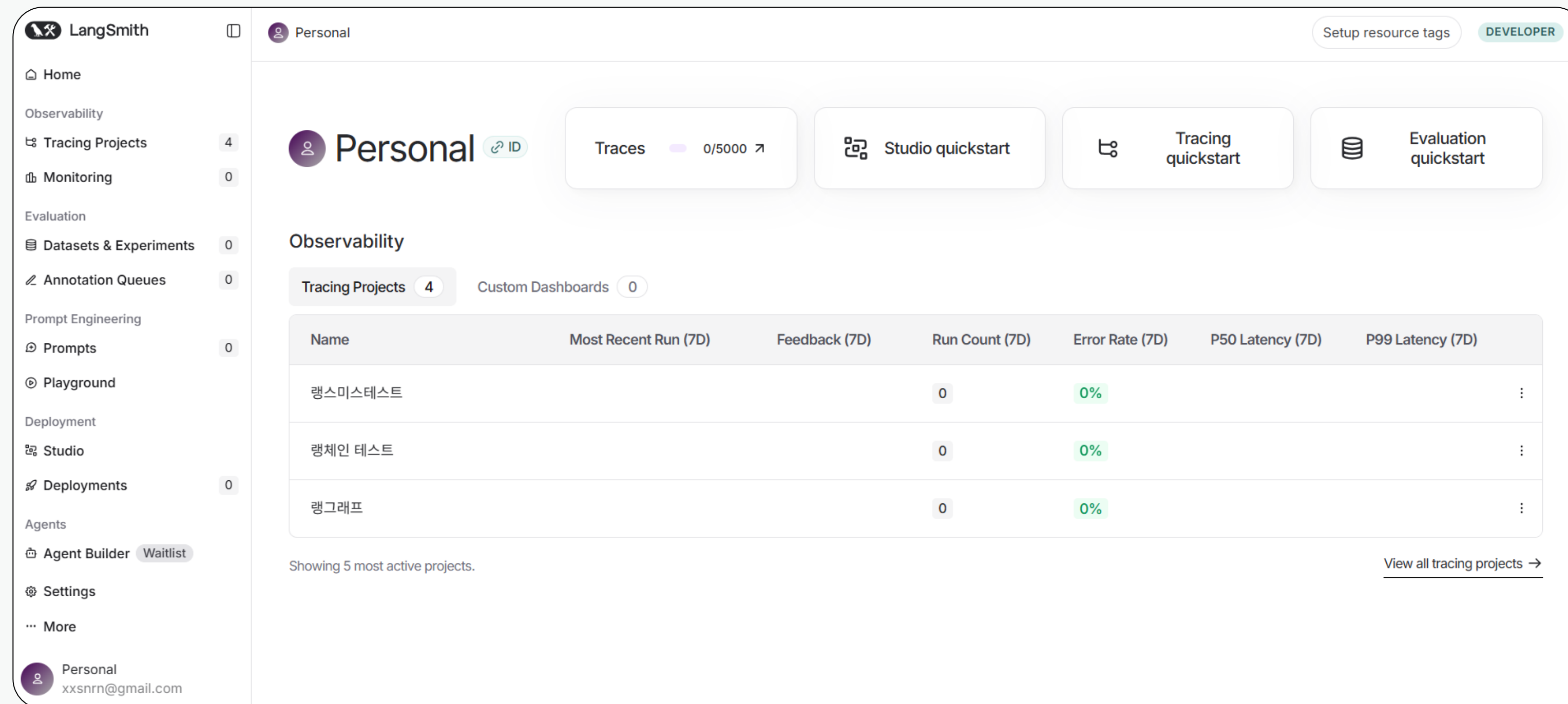
# Titanic 데이터셋을 기반으로 대화형 데이터 분석을 수행할 수 있는 CSV 에이전트를 생성합니다.
agent_executor = create_csv_agent(
    llm,                                # LLM(예: GPT 모델) 객체, 사용자 질문을 처리하고 응답 생성
    "titanic.csv",                     # 분석할 CSV 파일 경로 (예: Titanic 데이터셋)
    agent_type="openai-tools",         # 에이전트가 사용할 도구 유형 설정 (OpenAI의 함수 호출 방식과 호환)
    verbose=True,                      # 실행 과정을 상세히 출력하여 디버깅이나 학습에 도움
    allow_dangerous_code=True          # 코드 실행을 허용함 (주의: 잠재적으로 위험한 코드 실행 가능)
)
```


Langsmith





Langsmith란?



LangChain 애플리케이션 기반 추적·평가·모니터링할 수 있게 해주는 플랫폼

Langsmith 실행

1. 환경변수 설정

```
import os

os.environ["LANGSMITH_API_KEY"] = "lsv2_pt_1de3d72b0cec424e8a7b931e23853e3b_3a
os.environ["LANGSMITH_TRACING"] = "true"
os.environ["LANGSMITH_PROJECT"] = "POSCO"
os.environ["LANGSMITH_ENDPOINT"] = "https://api.smith.langchain.com"
os.environ["OPENAI_API_KEY"] = "sk-proj-H5kDxrmL87b0QeyMiC8JqAB6uvKih-o8m3qaH"
```

2. 체인 실행

```
chain = llm | parser
chain.invoke("겨울에 대해 설명해줘")
```




Langsmith 결과

TRACE

Waterfall

RunnableSequence

8.04s 436

ChatOpenAI

gpt-4o-mini

8.04s 436

Some runs have been hidden. [Show 1 hidden run](#)

RunnableSequence

Run Feedback Metadata

Input

Input

겨울에 대해 설명해줘

Output

Output

겨울은 사계절 중 하나로, 대개 12월에서 2월 사이에 해당하는 계절입니다. 이 시기는 북반구에서 가장 추운 시기로, 날씨가 차갑고 눈이 내리는 경우가 많습니다. 겨울의 주요 특징과 영향을 살펴보면 다음과 같습니다.

1. ****기온****: 겨울철에는 기온이 낮아지며, 특히 북극과 가까운 지역에서는 매우 극심한 추위가 발생합니다. 이로 인해 얼음과 눈이 형성됩니다.

Llama-Index



라마인덱스



외부 데이터를 손쉽게 LLM과 연결해 검색·질의·요약 등을 할 수 있게 해주는



랭체인 vs 라마인덱스

구분	LlamaIndex	LangChain
핵심 목적	외부 데이터를 LLM과 연결해 RAG 중심 기능 제공	LLM 기반 애플리케이션 전반 제작을 위한 프레임워크
데이터 처리 방식	Loader → Index → QueryEngine 중심 단일 흐름	Prompt, Retriever, Memory, Tool 등 모듈 조합형
사용 난이도	단순하고 빠르게 문서 기반 챗봇 구현 가능	복잡하지만 다양한 기능을 유연하게 구성 가능
확장성	인덱스 기반으로 문서·DB·API 등 연결 가능	에이전트, 멀티모달, 함수 호출 등 다양한 시나리오 지원
적합한 사용처	PDF/문서 QA, 내부 지식 조회 등 빠른 RAG 프로토타이핑	복잡한 LLM 앱 개발, 멀티 에이전트 시스템 구축

LLM & Embedding

OpenAI의 LLM과 임베딩 모델을 LlamaIndex에서 쉽게 쓸 수 있게 해주는 클래스

- 코드

```
from llama_index.llms.openai import OpenAI
from llama_index.embeddings.openai import OpenAIEmbedding

llm = OpenAI(model="gpt-4o-mini")
embedding = OpenAIEmbedding(model="text-embedding-3-large")
```


SimpleDirectoryReader

특정 폴더 안의 파일들을 읽어서 문서 리스트로 바꿔주는 도구

- 코드

```
from llama_index.core import SimpleDirectoryReader

# SimpleDirectoryReader를 사용하여 지정한 디렉토리 내의 모든 파일을 읽고
# 문서 리스트로 변환합니다. 여기서는 "/content" 폴더에 있는 파일들을 불러옵니다.
documents = SimpleDirectoryReader("/content").load_data()
```

VectorStoreIndex

문서를 벡터로 임베딩해서 유사도 기반 검색이 가능하도록 인덱스를 만들어주는 기능

- 코드

```
from llama_index.core import VectorStoreIndex

# 불러온 문서들(documents)에 대해 벡터 임베딩을 생성하고,
# 이를 기반으로 벡터 스토어 인덱스를 구축합니다.
# 이 인덱스는 RAG(Retrieval-Augmented Generation) 작업에서
# 유사도 기반 문서 검색 등을 수행할 수 있도록 해줍니다.

vector_index = VectorStoreIndex.from_documents(
    documents,          # 벡터화할 문서 리스트
    embed_model=embedding # 문서를 임베딩할 임베딩 모델 객체
)
```

as_query_engine()

인덱스를 자연어 질문을 처리할 수 있는 형태의 쿼리 엔진으로 변환

- 코드

```
# 생성된 벡터 인덱스를 기반으로 질의(Query)를 수행할 수 있는  
# 쿼리 엔진(query_engine)을 생성합니다.  
# 쿼리 엔진을 통해 사용자 입력에 맞는 관련 문서를 검색하고,  
# LLM과 함께 질의 응답을 수행할 수 있습니다.
```

```
query_engine = vector_index.as_query_engine()
```


query()

쿼리 엔진에 질문을 던져서 문서를 검색하고 답변을 생성하는 메서드

- 코드

```
# 쿼리 엔진(query_engine)을 사용하여 질문을 전달하고,  
# 해당 질문과 가장 관련성이 높은 문서를 검색한 뒤,  
# LLM을 통해 질의 응답을 수행합니다.  
# 아래 예시는 "넥스트몬의 신용등급"에 대한 정보를 질의하는 코드입니다.
```

```
response = query_engine.query("넥스트몬의 신용등급")  
print(response)
```

검색 생성 세부 설정





검색 생성 설정 비교

구분	as_query_engine()	Retriever + Synthesizer + Postprocessor
검색 제어 수준	기본 설정 사용	similarity_top_k, 검색 방식 등 상세 제어 가능
후처리 기능	불가	유사도 기준 필터링, 중복 제거 등 후처리 가능
응답 생성 방식	기본 요약 방식	refine, compact 등 다양한 응답 전략 선택 가능
확장성/유지보수성	제한적	검색, 후처리, 응답 생성 단계별 교체·확장 가능
적합한 사용 시점	간단한 테스트, 빠른 시작	프로덕션 수준, RAG 고도화 필요 시

VectorIndexRetriever

벡터 인덱스를 기반으로 사용자의 쿼리와 유사도가 높은 문서 조각을 검색하는 모듈

- 코드

```
from llama_index.core.retrievers import VectorIndexRetriever

# 벡터 기반 검색을 수행하는 Retriever 생성
# - index: 사용할 벡터 인덱스 (vector_index)
# - similarity_top_k: 유사도 기준 상위 3개의 노드를 검색
retriever = VectorIndexRetriever(index=vector_index, similarity_top_k=3)
```

get_response_synthesizer

검색된 문서 조각들을 종합해 자연어 형태의 응답을 생성하는 기능을 제공하는 도구

- 코드

```
from llama_index.core import get_response_synthesizer

# 검색된 노드들로부터 응답을 생성하는 응답 생성기
# - 검색 결과를 바탕으로 자연어 형태의 답변을 생성
response_synthesizer = get_response_synthesizer()
```


SimilarityPostprocessor

검색된 문서 조각 중 유사도 점수가 기준 이하인 결과를 필터링하는 후처리 모듈.

- 코드

```
from llama_index.core.postprocessor import SimilarityPostprocessor

# 후처리기 설정
# - similarity_cutoff: 유사도 점수가 0.3 미만인 노드들은 필터링 (제거)
postprocessor = SimilarityPostprocessor(similarity_cutoff=0.3)
```

RetrieverQueryEngine

검색기, 응답 생성기, 후처리를 결합하여 쿼리에 대한 최종 답변을 생성하는 상위 엔진.

- 코드

```
from llama_index.core.query_engine import RetrieverQueryEngine

# 최종 질의 엔진 구성
# - retriever: 문서 검색을 담당
# - response_synthesizer: 검색 결과로부터 답변 생성
# - node_postprocessors: 검색된 노드를 후처리(필터링, 정렬 등)하기 위한 모듈 목록
query_engine_tuned = RetrieverQueryEngine(
    retriever=retriever,
    response_synthesizer=response_synthesizer,
    node_postprocessors=[postprocessor]
)
```

RAG 실제 활용 예시: 산업별 적용

각 산업 분야별로 특화된 RAG 적용 사례와 효과



금융 산업

- ✓ 투자 보고서 자동 분석 및 요약
- ✓ 규제 준수 문서 검토 자동화
- ✓ 맞춤형 금융 자문 챗봇
- 💡 초당 수천 개의 금융 문서와 실시간 데이터 처리 가능



의료 산업

- ✓ 의학 논문 및 임상 지식 검색
- ✓ 환자 기록 기반 진단 보조
- ✓ 의약품 정보 및 상호작용 안내
- 💡 최신 의료 지식과 병원 내부 가이드라인 연계



법률 산업

- ✓ 판례 및 법령 검색 시스템
- ✓ 계약서 검토 및 법적 리스크 분석
- ✓ 소송 전략 수립 보조
- 💡 수십년 분량의 법률 자료 인덱싱 및 고정밀 검색



IT 산업

- ✓ 기술 문서 및 코드베이스 검색
- ✓ 지식 관리 및 기술 지원 시스템
- ✓ 코드 최적화 및 디버깅 보조
- 💡 Github, Stack Overflow와 사내 코드 통합 검색

RAG 평가

RAG의 평가는 단순히 생성된 답변의 품질뿐 아니라,모델이 정확한 정보를 얼마나 잘 검색하고, 그 정보를 얼마나 잘 활용했는지를 함께 측정합니다.

평가 단계	설명	대표 지표
Retrieval 평가	주어진 질문에 대해 얼마나 관련성 높은 문서를 잘 찾아냈는가	Recall@K, Precision@K, MRR, nDCG
Generation 평가	검색된 문서를 바탕으로 생성된 답변의 품질	BLEU, ROUGE, METEOR, BERTScore
End-to-End 평가	최종 답변이 사용자의 질문에 얼마나 정확하고 유용한가	Faithfulness, Relevance, Factuality, Answer Correctness

챗봇 스트림릿 페이지 코드 생성

무엇을 도와드릴까요?

```
llm = ChatOpenAI( model="gpt-4o-mini")  
parser = StrOutputParser()
```

```
chain = prompt | llm | parser
```

위 코드 기반의 챗봇 스트림릿 코드 만들어줘

+



GPT에게 랭체인 또는 라마인덱스 코드를 가져와서 스트림릿 코드 생성 요청



챗봇 스트림릿 페이지 코드 생성



app.py

python

코드 복사

```
import streamlit as st
from langchain_core.output_parsers import StrOutputParser
from langchain_core.prompts import ChatPromptTemplate
from langchain_openai import ChatOpenAI

# 1 페이지 기본 설정
st.set_page_config(page_title="LangChain Chatbot", page_icon="🤖")
st.title("💬 데이터 전문가 챗봇")

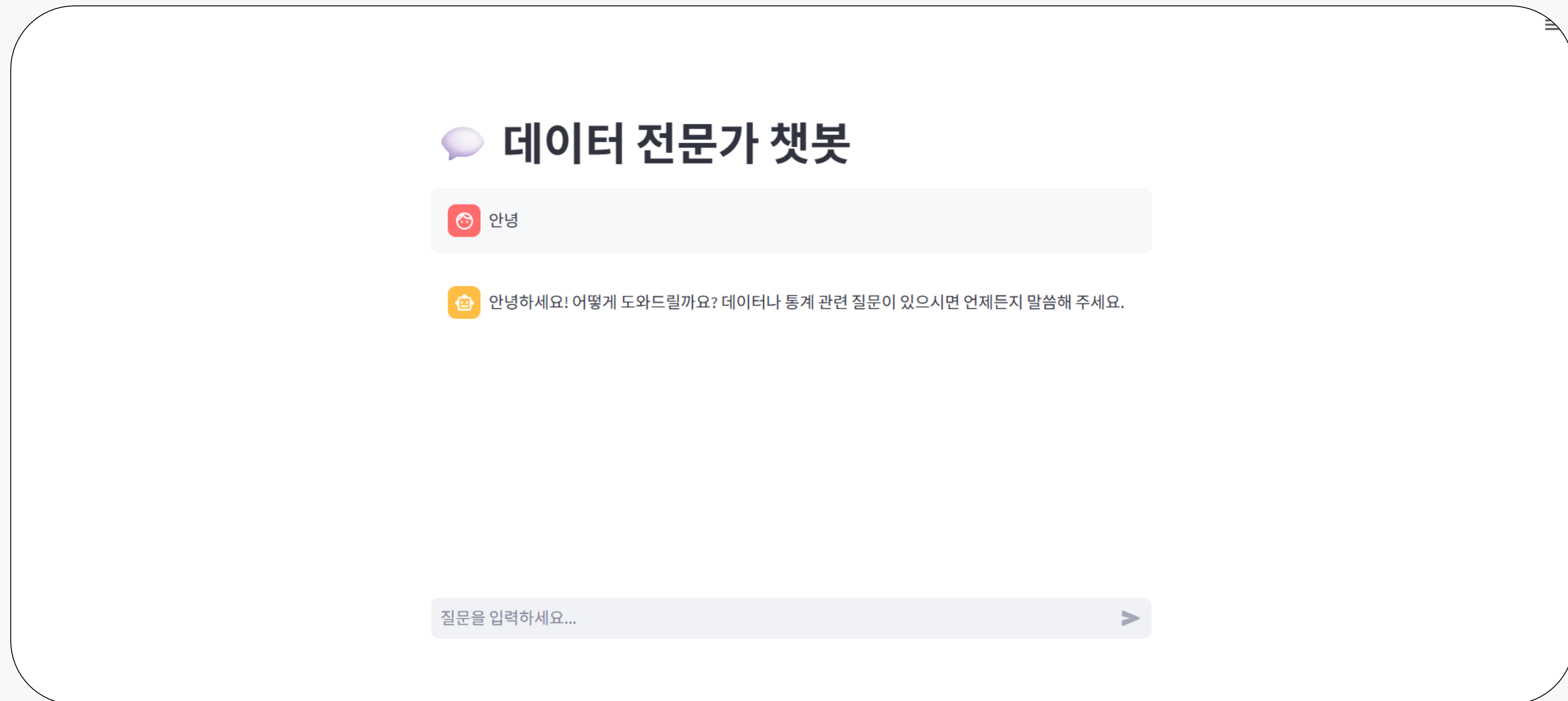
# 2 메시지 히스토리 초기화
if "messages" not in st.session_state:
    st.session_state["messages"] = []
```



생성된 코드 확인



챗봇 스트림릿 페이지 코드 생성



챗봇 스트림릿 페이지 확인



챗봇 스트림릿 페이지 코드 생성

✕

⋮

⚙️ **설정**

모델 선택

gpt-4o-mini

▼

창의성 (Temperature)

0.70

0.001.00

최대 토큰 수

500

502000

📄 대화 내보내기 (CSV)

🗑️ 대화 초기화

💬 **LangChain 데이터 전문가 챗봇**

💬 **대화**

🗣️ You: ai에 대해 알려줘

🛒 AI: AI(인공지능)는 컴퓨터 시스템이 인간의 지능을 모방하여 학습, 추론, 문제 해결, 이해 및 언어 처리 등의 작업을 수행할 수 있도록 하는 기술입니다. AI는 크게 두 가지 범주로 나눌 수 있습니다:

1. **협정의 AI (Narrow AI):** 특정 작업이나 문제에 특화된 AI입니다. 예를 들어, 이미지 인식, 자연어 처리, 추천 시스템 등이 이에 해당합니다. 현재 대부분의 상용 AI 시스템이 이 범주에 포함됩니다.

2. **광의의 AI (General AI):** 인간처럼 다양한 지적 작업을 수행할 수 있는 AI로, 아직 실현되지 않았습니다. 이 AI는 모든 종류의 문제를 이해하고 해결할 수 있는 능력을 가지고 있어야 합니다.

AI 기술은 다양한 알고리즘과 모델을 사용하여 발전해왔습니다. 그 중에서도 다음과 같은 기술이 중요합니다:

질문을 입력하세요... ➤

GPT에게 부가 기능들을 추가 요청



POSCO GROUP
AI EXPERT COURSE